Language Fundamentals

Introduction

- Python is a general purpose high level programming language.
- Python was developed by Guido Van Rossam in 1989 while working at National Research Institute at Netherlands.
- But officially Python was made available to public in 1991. The official Date of Birth for Python is : Feb 20th 1991.
- Python is recommended as first programming language for beginners.

Eg1: To print

Helloworld: Java:

```
1) public class HelloWorld
2) {
3)                 p s v main(String[] args)
4)    {
5)                    SOP("Hello world");
6)    }
7) }
```

C:

```
1) #include<stdio.h>
2) void main()
3) {
4)     print("Hello world");
5) }
```

Python:

```
print("Hello World")
```

Eg2: To print the sum of 2 numbers

Java:

```
1)  public class Add
2)  {
3)      public static void main(String[] args)
4)  {
5)          int a,b;
6)          a =10;
7)          b=20;
8)          System.out.println("The Sum:"+(a+b));
9)  }
10) }
```

C:

```
1)  #include <stdio.h>
2)
3) void main()
4) {

6)      a =10;

8)      printf("The Sum:%d",(a+b));
7)      b=20;
```

Python:

```
1)  a=10
2)  b=20
3)  print("The Sum:",(a+b))
```

The name Python was selected from the TV
Show "The Complete

Monty
Python's

Circus", which was broadcasted in BBC from 1969 to 1974.

Guido developed Python language by taking almost all programming features from different languages

1. Functional Programming Features from C

2. Object Oriented Programming Features from C++
3. Scripting Language Features from Perl and Shell Script

4. Modular Programming Features from Modula-3

Most of syntax in Python Derived from C and ABC

languages. Where we can use Python:

We can use everywhere. The most common important application areas are

1. For developing Desktop Applications
2. For developing web Applications
3. For developing database Applications
4. For Network Programming
5. For developing games
6. For Data Analysis Applications
7. For Machine Learning
8. For developing Artificial Intelligence Applications
9. For IOT

...

Note:

Internally Google and Youtube use Python coding

NASA and Nework Stock Exchange Applications developed by Python.

Top Software companies like Google, Microsoft, IBM, Yahoo using Python.

Features of Python:

1. Simple and easy to learn:

Python is a simple programming language. When we read Python program,we can feel like reading english statements.

The syntaxes are very simple and only 30+ kerywords are available.

When compared with other languages, we can write programs with very less number of lines. Hence more readability and simplicity.

We can reduce development and cost of the project.

2. Freeware and Open Source:

We can use Python software without any licence and it is freeware.

Its source code is open,so that we can we can customize based on our

requirement. Eg: Jython is customized version of Python to work with Java Applications.

3. High Level Programming language:

Python is high level programming language and hence it is programmer friendly language. Being a programmer we are not required to concentrate low level activities like memory management and security etc..

4. Platform Independent:

Once we write a Python program,it can run on any platform without rewriting once again. Internally PVM is responsible to convert into machine understandable form.

5. Portability:

Python programs are portable. ie we can migrate from one platform to another platform very easily. Python programs will provide same results on any paltform.

6. Dynamically Typed:

In Python we are not required to declare type for variables. Whenever we are assigning the value, based on value, type will be allocated automatically.Hence Python is considered as dynamically typed language.

But Java, C etc are Statically Typed Languages b'z we have to provide type at the beginning only.

This dynamic typing nature will provide more flexibility to the programmer.

7. Both Procedure Oriented and Object Oriented:

Python language supports both Procedure oriented (like C, pascal etc) and object oriented (like C++,Java) features. Hence we can get benefits of both like security and reusability etc

8. Interpreted:

We are not required to compile Python programs explcitly. Internally Python interpreter will take care that compilation.

If compilation fails interpreter raised syntax errors. Once compilation success then PVM (Python Virtual Machine) is responsible to execute.

9. Extensible:

We can use other language programs in Python. The main advantages of this approach are:

1. We can use already existing legacy non-Python code
2. We can improve performance of the application

10. Embedded:

We can use Python programs in any other language programs.

i.e we can embedd Python programs anywhere.

11. Extensive Library:

Python has a rich inbuilt library.

Being a programmer we can use this library directly and we are not responsible to implement the functionality.

etc...

Limitations of Python:

1. Performance wise not up to the mark b'z it is interpreted language.
2. Not using for mobile Applications

Flavors of Python:

1. CPython:
It is the standard flavor of Python. It can be used to work with C lanugage Applications

2. Jython or JPython:
It is for Java Applications. It can run on JVM

3. IronPython:
It is for C#.Net platform

4. PyPy:
The main advantage of PyPy is performance will be improved because JIT compiler is available inside PVM.

5. RubyPython
For Ruby Platforms

6. AnacondaPython
It is specially designed for handling large volume of data processing.

...

Python Versions:

Python 1.0V introduced in Jan 1994
Python 2.0V introduced in October
2000 Python 3.0V introduced in
December 2008

Note: Python 3 won't provide backward compatibility to Python2

i.e there is no guarantee that Python2 programs will run in Python3.

Current versions

Python 3.6.1	Python 2.7.13

Identifiers

A name in Python program is called identifier.

It can be class name or function name or module name or variable

name. a = 10

Rules to define identifiers in Python:

1. The only allowed characters in Python are

- alphabet symbols(either lower case or upper case)
- digits(0 to 9)
- underscore symbol(_)

By mistake if we are using any other symbol like $ then we will get syntax error.

- cash = 10 √
- ca$h =20

2. Identifier should not starts with digit

- 123total
- total123 √

3. Identifiers are case sensitive. Of course Python language is case sensitive language.

- total=10
- TOTAL=999
- print(total) #10
- print(TOTAL) #999

Identifier:

1. Alphabet Symbols (Either Upper case OR Lower case)

2. If Identifier is start with Underscore (_) then it indicates it is private.

3. Identifier should not start with Digits.

4. Identifiers are case sensitive.

5. We cannot use reserved words as identifiers Eg: def=10

6. There is no length limit for Python identifiers. But not recommended to use too lengthy identifiers.

7. Dollor ($) Symbol is not allowed in Python.

Q. Which of the following are valid Python identifiers?

1) 123total
2) total123 √
3) java2share √
4) ca$h
5) _abc_abc_ √
6) def
7) if

Note:

1. If identifier starts with _ symbol then it indicates that it is private
2. If identifier starts with __(two under score symbols) indicating that strongly private identifier.
3. If the identifier starts and ends with two underscore symbols then the identifier is language defined special name,which is also known as magic methods.

Eg: add

Reserved Words

In Python some words are reserved to represent some meaning or functionality. Such type of words are called Reserved words.

There are 33 reserved words available in Python.

- True,False,None
- and, or ,not,is
- if,elif,else
- while,for,break,continue,return,in,yield
- try,except,finally,raise,assert
- import,from,as,class,def,pass,global,nonlocal,lambda,del,with

Note:

1. All Reserved words in Python contain only alphabet symbols.

2. Except the following 3 reserved words, all contain only lower case alphabet symbols.

- True
- False
- None

Eg: a= true

a=True ✓

```
>>> import keyword
>>> keyword.kwlist
['False', 'None', 'True', 'and', 'as', 'assert', 'break', 'class', 'continue', 'def', 'del', 'elif', 'else',
'except', 'finally', 'for', 'from', 'global', 'if', 'import', 'in', 'is', 'lambda', 'nonlocal', 'not', 'or',
'pass', 'raise', 'return', 'try', 'while', 'with', 'yield']
```

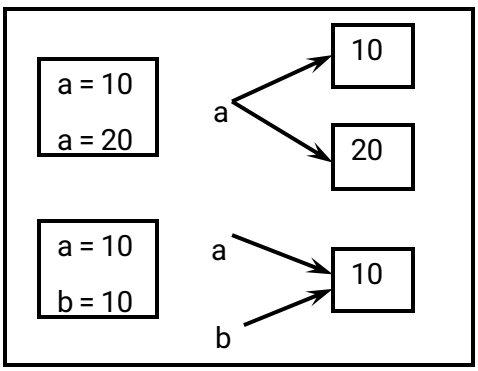
Data Types

Data Type represent the type of data present inside a variable.

In Python we are not required to specify the type explicitly. Based on value provided,the type will be assigned automatically.Hence Python is Dynamically Typed Language.

Python contains the following inbuilt data types

1. int
2. float
3. complex
4. bool
5. str
6. bytes
7. bytearray
8. range
9. list
10. tuple
11. set
12. frozen set
13. dict
14. None



Note: Python contains several inbuilt functions

1. type()
   to check the type of variable

2. id()
   to get address of object

3. print()
to print the value

In Python everything is object

int data type:

We can use int data type to represent whole numbers (integral values) Eg:

```
a=10
type(a) #int
```

Note:

In Python2 we have long data type to represent very large integral values.

But in Python3 there is no long type explicitly and we can represent long values also by using int type only.

We can represent int values in the following ways

1. Decimal form
2. Binary form
3. Octal form
4. Hexa decimal form

1. Decimal form(base-10):

It is the default number system in Python The allowed digits are: 0 to 9

Eg: a =10

2. Binary form(Base-2):

The allowed digits are : 0 & 1

Literal value should be prefixed with 0b or 0B

Eg: a =
0B1111 a
=0B123

a=b111

3. Octal Form(Base-8):

The allowed digits are : 0 to 7

Literal value should be prefixed with 0o or 0O.

Eg: a=0o123

a=0o786

4. Hexa Decimal Form(Base-16):

The allowed digits are : 0 to 9, a-f (both lower and upper cases are allowed)
Literal value should be prefixed with 0x or 0X

Eg:

a =0XFACE

a=0XBeef

a
=0XBeer

Note: Being a programmer we can specify literal values in decimal, binary, octal and hexa decimal forms. But PVM will always provide values only in decimal form.

a=10

b=0o1
0
c=0X1
0
d=0B1
0

print(a)10
print(b)8
print(c)16
print(d)2

Base Conversions

Python provide the following in-built functions for base conversions

1. bin():

We can use bin() to convert from any base to

binary Eg:

```
1)  >>> bin(15)
2)  '0b1111'
3)  >>> bin(0o11)
4)  '0b1001'
```

```
5) >>> bin(0X10)
6) '0b10000'
```

2. oct():

We can use oct() to convert from any base to octal

Eg:

```
1)  >>> oct(10)
2)  '0o12'
3)  >>> oct(0B1111)
4)  '0o17'
5)  >>> oct(0X123)
6)   '0o443'
```

3. hex():

We can use hex() to convert from any base to hexa

decimal Eg:

```
1)  >>> hex(100)
2)  '0x64'
3)  >>> hex(0B111111)
4)  '0x3f'
5)  >>> hex(0o12345)
6)   '0x14e5'
```

float data type:

We can use float data type to represent floating point values (decimal values)

Eg: f=1.234

type(f) float

We can also represent floating point values by using exponential form (scientific notation)

Eg: f=1.2e3

print(f) 1200.0

instead of 'e' we can use 'E'

The main advantage of exponential form is we can represent big values in less memory.

***Note:

We can represent int values in decimal, binary, octal and hexa decimal forms. But we can represent float values only by using decimal form.

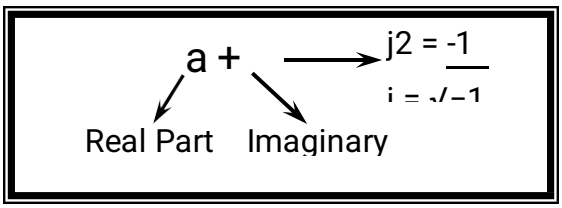
Eg:

```
1)>>> f=0B11.01
  2)   File "<stdin>", line 1
  3)             f=0B11.01
```

```
4)          ^
5) SyntaxError: invalid syntax
6)
7) >>> f=0o123.456
8)   SyntaxError: invalid syntax
9)
10) >>> f=0X123.456
11) SyntaxError: invalid syntax
```

Complex Data Type:

A complex number is of the form



a and b contain intergers or floating point

values Eg:

3+5j

10+5.5j

0.5+0.1j

In the real part if we use int value then we can specify that either by decimal,octal,binary or hexa decimal form.

But imaginary part should be specified only by using decimal form.

```
1) >>> a=0B11+5j
2) >>> a
3) (3+5j)
4) >>> a=3+0B15j
5) SyntaxError: invalid syntax
```

Even we can perform operations on complex type values.

```
1) >>> a=10+1.5j
2) >>> b=20+2.5j
3) >>> c=a+b
4) >>> print(c)
5) (30+4j)
6) >>> type(c)
7) <class 'complex'>
```

Note: Complex data type has some inbuilt attributes to retrieve the real part and imaginary part

c=10.5+3.6j

c.real==>10.5
c.imag==>3.6

We can use complex type generally in scientific Applications and electrical engineering Applications.

4.bool data type:

We can use this data type to represent boolean values. The only allowed values for this data type are:

True and False

Internally Python represents True as 1 and False

as 0 b=True

type(b) =>bool

Eg:

a=10
b=20

c=a<b
print(c)==>Tru
e

True+True==>
2 True-
False==>1

str type:

str represents String data type.

A String is a sequence of characters enclosed within single quotes or double quotes.

s1='durga'

s1="durga"

By using single quotes or double quotes we cannot represent multi line string literals.

s1="nitin

Gandhi”

For this requirement we should go for triple single quotes(''') or triple double quotes(""")

s1='''nitin

gandhi'''

s1="""nitin

gandhi"""

We can also use triple quotes to use single quote or double quote in our

String. ''' This is " character'''

' This i " Character '

We can embed one string in another string

'''This "Python class very helpful" for java students'''

Slicing of Strings:

slice means a piece

[ ] operator is called slice operator,which can be used to retrieve parts of String. In Python Strings follows zero based index.

The index can be either +ve or -ve.

+ve index means forward direction from Left to Right

-ve index means backward direction from Right to Left

	-5	-4	-3	-2	-1
	d	u	r	g	a
	0	1	2	3	4

1) >>> s="durga"

```
2   >>> s[0]
)
3   'd'
)
4   >>> s[1]
)
5   'u'
)
6   >>> s[-1]
)
7   'a'
)
8   >>> s[40]
)
```

IndexError: string index out of range

```
1)  >>> s[1:40]
2) 'urga'
3) >>> s[1:]
4) 'urga'
5) >>> s[:4]
6) 'durg'
7) >>> s[:]
8) 'durga'
9) >>>
10)
11)
12) >>> s*3
13) 'durgadurgadurga'
14)
15) >>> len(s)
16) 5
```

Note:

1. In Python the following data types are considered as Fundamental Data types

   - int
   - float
   - complex
   - bool
   - str

2. In Python,we can represent char values also by using str type and explicitly char type is not available.

Eg:

```
1)  >>> c='a'
2) >>> type(c)
3) <class 'str'>
```

3. long Data Type is available in Python2 but not in Python3. In Python3 long values also we can represent by using int type only.

4. In Python we can present char Value also by using str Type and explicitly char Type is not available.

Type Casting

We can convert one type value to another type. This conversion is called Typecasting or Type coersion.

The following are various inbuilt functions for type casting.

1. int()
2. float()
3. complex()
4. bool()
5. str()

1. int():

We can use this function to convert values from other types to

int Eg:

```
>>> int(123.987)
123
>>> int(10+5j)
TypeError: can't convert complex to int
>>> int(True)
1
>>> int(False)
0
>>> int("10")
10
>>> int("10.5")
ValueError: invalid literal for int() with base 10: '10.5'
>>> int("ten")
ValueError: invalid literal for int() with base 10: 'ten'
>>> int("0B1111")
ValueError: invalid literal for int() with base 10: '0B1111'
```

Note:

1. We can convert from any type to int except complex type.
2. If we want to convert str type to int type, compulsary str should contain only integral value and should be specified in base-10

2. float():

We can use float() function to convert other type values to float type.

```
>>> float(10)
10.0
>>> float(10+5j)
TypeError: can't convert complex to float
>>> float(True)
1.0
>>> float(False)
0.0
>>> float("10")
10.0
>>> float("10.5")
10.5
>>> float("ten")
ValueError: could not convert string to float: 'ten'
>>> float("0B1111")
ValueError: could not convert string to float: '0B1111'
```

Note:

1. We can convert any type value to float type except complex type.

2. Whenever we are trying to convert str type to float type compulsary str should be either integral or floating point literal and should be specified only in base-10.

3. complex():

We can use complex() function to convert other types to complex type.

Form-1: complex(x)

We can use this function to convert x into complex number with real part x and imaginary part 0.

Eg:

```
complex(10)==>10+0j
   complex(10.5)===>10.5+0j
   complex(True)==>1+0j
   complex(False)==>0j
   complex("10")==>10+0j
   complex("10.5")==>10.5+0j
complex("ten")
      ValueError: complex() arg is a malformed string
```

Form-2: complex(x,y)

We can use this method to convert x and y into complex number such that x will be real part and y will be imaginary part.

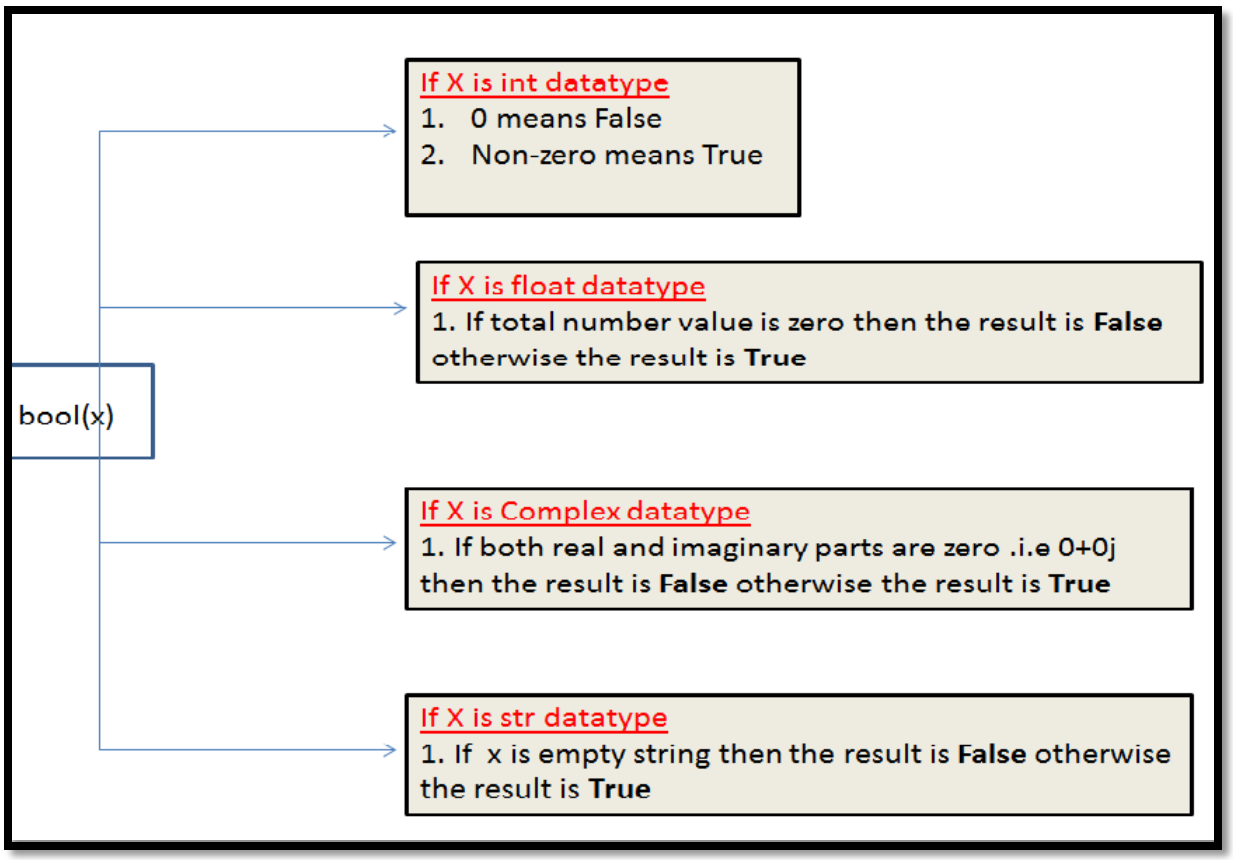
Eg: complex(10,-2)==>10-2j
complex(True,False)==>1+0j

4. bool():

We can use this function to convert other type values to bool

type. Eg:

```
1) bool(0)==>False
2) bool(1)==>True
3) bool(10)===>True
4) bool(10.5)===>True
5) bool(0.178)==>True
6) bool(0.0)==>False
7) bool(10-2j)==>True
8) bool(0+1.5j)==>True
9) bool(0+0j)==>False
10) bool("True")==>True
11) bool("False")==>True
12) bool("")==>False
```



5. str():

We can use this method to convert other type values to str

type Eg:

```
1)  >>> str(10)
2)  '10'
3)  >>> str(10.5)
4)  '10.5'
5)  >>> str(10+5j)
6)  '(10+5j)'
7)  >>> str(True)
8)   'True'
```

Fundamental Data Types vs Immutability:

All Fundamental Data types are immutable. i.e once we creates an object,we cannot perform any changes in that object. If we are trying to change then with those changes a new object will be created. This non-chageable behaviour is called immutability.

In Python if a new object is required, then PVM wont create object immediately. First it will check is any object available with the required content or not. If available then existing object will be reused. If it is not available then only a new object will be created. The advantage of this approach is memory utilization and performance will be improved.

But the problem in this approach is,several references pointing to the same object,by using one reference if we are allowed to change the content in the existing object then the remaining references will be effected. To prevent this immutability concept is required.

According to this once creates an object we are not allowed to change content. If we are trying to change with those changes a new object will be created.

Eg:

```
1)  >>> a=10
2) >>> b=10
3) >>> a is b
4) True
5) >>> id(a)
6) 1572353952
7) >>> id(b)
8) 1572353952
9) >>>
```

>>> a=10
>>> b=10
>>> id(a)
1572353952
>>> id(b)
1572353952
>>> a is b
True
	>>> a=10+5j
	>>> b=10+5j
	>>> a is b
	False
	>>> id(a)
	15980256
	>>> id(b)
	15979944
		>>> a=True
		>>> b=True
		>>> a is b
		True
		>>> id(a)
		1572172624
		>>> id(b)
		1572172624
			>>> a='durga'
			>>> b='durga'
			>>> a is b
			True
			>>> id(a)
			1637884
			8
			>>> id(b)

bytes Data Type:

bytes data type represens a group of byte numbers just like an

array. Eg:

```
1)   x =
2)      b =
3)      type(b)==>byte
4)      print(b[0])==> 10
5)      print(b[-1])==> 40
6)      >>> for i in b : print(i)
7)
8)        10
          20
10        30
11        40
```

Conclusion 1:

The only allowed values for byte data type are 0 to 256. By mistake if we are trying to provide any other values then we will get value error.

Conclusion 2:

Once we creates bytes data type value, we cannot change its values,otherwise we will get TypeError.

Eg:

```
1)  >>> x=[10,20,30,40]
2)  >>> b=bytes(x)
3)  >>> b[0]=100
4)    TypeError: 'bytes' object does not support item assignment
```

bytearray Data type:

bytearray is exactly same as bytes data type except that its elements can be

modified. Eg 1:

```
1)  x=[10,20,30,40]
2)  b = bytearray(x)
3)  for i in b : print(i)
4)  10
```

```
5) 20
6) 30
7) 40
8)    b[0]=100
9)                  for i in b: print(i)
10) 100
11) 20
12) 30
13) 40
```

Eg 2:

```
1)  >>> x =[10,256]
2)  >>> b = bytearray(x)
3)  ValueError: byte must be in range(0, 256)
```

list data type:

If we want to represent a group of values as a single entity where insertion order required to preserve and duplicates are allowed then we should go for list data type.

1. insertion order is preserved
2. heterogeneous objects are allowed
3. duplicates are allowed
4. Growable in nature
5. values should be enclosed within square brackets.

Eg:

```
1)  list=[10,10.5,'durga',True,10]
2)      print(list) # [10,10.5,'durga',True,10]
```

Eg:

```
1)  list=[10,20,30,40]
2)  >>> list[0]
3)  10
4)  >>> list[-1]
5)  40
6)  >>> list[1:3]
7)  [20, 30]
8)  >>> list[0]=100
9)  >>> for i in list:print(i)
10) ...
11) 100
12) 20
13) 30
```

```
14) 40
```

list is growable in nature. i.e based on our requirement we can increase or decrease the size.

```
1)  >>> list=[10,20,30]
2) >>> list.append("durga")
3) >>> list
4) [10, 20, 30, 'durga']
5) >>> list.remove(20)
6) >>> list
7) [10, 30, 'durga']
8) >>> list2=list*2
9) >>> list2
10) [10, 30, 'durga', 10, 30, 'durga']
```

Note: An ordered, mutable, heterogenous collection of eleemnts is nothing but list, where duplicates also allowed.

tuple data type:

tuple data type is exactly same as list data type except that it is immutable.i.e we cannot chage values.

Tuple elements can be represented within

parenthesis. Eg:

```
1)t=(10,20,30,40)
2)   type(t)
3) <class 'tuple'>
4)   t[0]=100
5) TypeError: 'tuple' object does not support item assignment
6) >>> t.append("durga")
7) AttributeError: 'tuple' object has no attribute 'append'
8) >>> t.remove(10)
9) AttributeError: 'tuple' object has no attribute 'remove'
```

Note: tuple is the read only version of list

range Data Type:

range Data Type represents a sequence of numbers.

The elements present in range Data type are not modifiable. i.e range Data type is immutable.

Form-1: range(10)

generate numbers from 0 to 9

Eg:

r=range(10)

for i in r : print(i) 0 to 9

Form-2: range(10,20)

generate numbers from 10 to

19 r = range(10,20)

for i in r : print(i) 10 to 19

Form-3: range(10,20,2)

2 means increment

value r = range(10,20,2)

for i in r : print(i) 10,12,14,16,18

We can access elements present in the range Data Type by using index.
r=range(10,20)

r[0]==>10

r[15]==>IndexError: range object index out of

range We cannot modify the values of range

data type

Eg:

r[0]=100

TypeError: 'range' object does not support item

assignment We can create a list of values with range

data type

Eg:

```
1)  >>> l = list(range(10))
2)  >>> l
```

3) [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

set Data Type:

If we want to represent a group of values without duplicates where order is not important then we should go for set Data Type.

1. insertion order is not preserved
2. duplicates are not allowed
3. heterogeneous objects are allowed
4. index concept is not applicable
5. It is mutable collection
6. Growable in nature Eg:

```
1)s={100,0,10,200,10,'durga'}
2)   s # {0, 100, 'durga', 200, 10}
3) s[0] ==>TypeError: 'set' object does not support indexing
4)
5) set is growable in nature, based on our requirement we can increase or decrease the size.
6)
7) >>> s.add(60)
8) >>> s
9) {0, 100, 'durga', 200, 10, 60}
10) >>> s.remove(100)
11) >>> s
12) {0, 'durga', 200, 10, 60}
```

frozenset Data Type:

It is exactly same as set except that it is immutable. Hence we cannot use add or remove functions.

```
1)  >>> s={10,20,30,40}
2) >>> fs=frozenset(s)
3) >>> type(fs)
4) <class 'frozenset'>
5) >>> for i in fs:print(i)
6) ...
7) 40
8) 10
9) 20
10) 30
11) >>> fs.add(70)
12) Attribute Error: 'frozenset' object has no attribute 'add'
13) >>> fs.remove(10)
14) Attribute Error: 'frozenset' object has no attribute 'remove'
```

dict Data Type:

If we want to represent a group of values as key-value pairs then we should go for dict data type.

Eg:

d={101:'durga',102:'ravi',103:'shiva'}

Duplicate keys are not allowed but values can be duplicated. If we are trying to insert an entry with duplicate key then old value will be replaced with new value.

Eg:

```
>>> d={101:'durga',102:'ravi',103:'shiva'}
>>> d[101]='sunny'
>>> d
{101: 'sunny', 102: 'ravi', 103: 'shiva'}

We can create empty dictionary as follows
 d={}
We can add key-value pairs as follows
d['a']='apple'
d['b']='banana'
print(d)
```

Note: dict is mutable and the order wont be preserved.

Note:

1. In general we can use bytes and bytearray data types to represent binary information like images,video files etc
2. In Python2 long data type is available. But in Python3 it is not available and we can represent long values also by using int type only.
3. In Python there is no char data type. Hence we can represent char values also by using str type.

Summary of Datatypes in Python3

Datatype	Description	Is Immutable	Example

Int	We can use to represent the whole/integral numbers	Immutable	>>> a=10
			>>> type(a)
			<class 'int'>

Float	We can use to represent the decimal/floating point numbers	Immutable	>>> b=10.5
			>>> type(b)
			<class 'float'>

Complex	We can use to represent the complex numbers	Immutable	>>> c=10+5j
			>>> type(c)
			<class 'complex'>
			>>>
			c.real
			10.0
			>>>
			c.imag
			5.0

Bool	We can use to represent the logical values(Only allowed values are True and False)	Immutable	>>> flag=True
			>>> flag=False
			>>> type(flag)
			<class 'bool'>

Str	To represent sequence of Characters	Immutable	>>> s='durga'
			>>> type(s)
			<class 'str'>
			>>> s="durga"
			>>> s='''Durga Software Solutions
			... Ameerpet'''
			>>> type(s)
			<class 'str'>

bytes	To represent a sequence of byte values from 0-255	Immutable	>>> list=[1,2,3,4]
			>>> b=bytes(list)
			>>> type(b)
			<class 'bytes'>

bytearray	To represent a sequence of byte values from 0-255	Mutable	>>> list=[10,20,30]
			>>> ba=bytearray(list)
			>>> type(ba)
			<class 'bytearray'>

range	To represent a range of values	Immutable	>>> r=range(10)
			>>> r1=range(0,10)
			>>> r2=range(0,10,2)

list	To represent an ordered collection of objects	Mutable	>>> l=[10,11,12,13,14,15]
			>>> type(l)
			<class 'list'>

tuple	To represent an ordered collections of objects	Immutable	>>> t=(1,2,3,4,5)
			>>> type(t)
			<class 'tuple'>

set	To represent an unordered collection of unique objects	Mutable	>>> s={1,2,3,4,5,6} >>> type(s)

frozenset	To represent an unordered collection of unique objects	Immutable	<class 'set'>
			>>> s={11,2,3,'Durga',100,'Ramu'}
			>>> fs=frozenset(s)
			>>> type(fs)
			<class 'frozenset'>

dict	To represent a group of key value pairs	Mutable	>>> d={101:'durga',102:'ramu',103:'hari'}
			>>> type(d)
			<class 'dict'>

None Data Type:

None means Nothing or No value associated.

If the value is not available,then to handle such type of cases None introduced. It is something like null value in Java.

Eg:

```
def m1():
  a=10

print(m1())
  None
```

Escape Characters:

In String literals we can use esacpe characters to associate a special meaning.

```
1)  >>> s="durga\nalwar"
2)  >>> print(s)
3)  durga
4)  alwar
5)  >>> s="durga\talwar"
6)  >>> print(s)
7)  durga    alwar
8)  >>> s="This is "symbol"
9)    File "<stdin>", line 1
10)     s="This is " symbol" ^
11) SyntaxError: invalid syntax
12) >>> s="This is \"symbol"
13) >>> print(s)
14) This is "symbol
15)
```

The following are various important escape characters in Python

1) \n==>New Line
2) \t===>Horizontal tab
3) \r ==>Carriage Return
4) \b===>Back space
5) \f===>Form Feed
6) \v==>Vertical tab
7) \'===>Single quote
8) \"===>Double quote
9) \\===>back slash symbol
   ....

Constants:

Constants concept is not applicable in Python.

But it is convention to use only uppercase characters if we don't want to change value.

MAX_VALUE=10

It is just convention but we can change the value.

Operators

Operator is a symbol that performs certain operations.
Python provides the following set of operators

1. Arithmetic Operators
2. Relational Operators or Comparison Operators
3. Logical operators
4. Bitwise oeprators
5. Assignment operators
6. Special operators

1. Arithmetic Operators:

+ ==>Addition

- ==>Subtraction

* ==>Multiplication

/ ==>Division operator

% ===>Modulo operator

// ==>Floor Division operator

** ==>Exponent operator or power

operator Eg: test.py:

```
1) a=10
2) b=2
3) print('a+b=',a+b)
4) print('a-b=',a-b)
5) print('a*b=',a*b)
6) print('a/b=',a/b)
7) print('a//b=',a//b)
8) print('a%b=',a%b)
9) print('a**b=',a**b)
```

Output:

```
1) Python test.py or py test.py
2  a+b= 12
)
3  a-b= 8
)
4) a*b= 20
5  a/b= 5.0
)
6) a//b= 5
7  a%b= 0
)
8) a**b= 100
```

Eg:

```
1) a = 10.5
2  b=2
)
3)
4  a+b= 12.5
)
5  a-b= 8.5
)
6  a*b= 21.0
)
7  a/b= 5.25
)
8  a//b= 5.0
)
9  a%b= 0.5
)
10) a**b= 110.25
```

Eg:

10/2==>5.0

10//2==>5

10.0/2===>5.0

10.0//2===>5.0

Note: / operator always performs floating point arithmetic. Hence it will always returns float value.

But Floor division (//) can perform both floating point and integral arithmetic. If arguments are int type then result is int type. If atleast one argument is float type then result is float type.

Note:

We can use +,* operators for str type also.

If we want to use + operator for str type then compulsory both arguments should be str type only otherwise we will get error.

```
1) >>> "durga"+10
2) TypeError: must be str, not int
3) >>> "durga"+"10"
4) 'durga10'
```

If we use * operator for str type then compulsory one argument should be int and other argument should be str type.

2*"durga"
"durga"*2

2.5*"durga" ==>TypeError: can't multiply sequence by non-int of type 'float'
"durga"*"durga"==>TypeError: can't multiply sequence by non-int of type 'str'

+=====>String concatenation operator

* ===>String multiplication operator

Note: For any number x,

x/0 and x%0 always raises "ZeroDivisionError"

10/0

10.0/0

.....

Relational Operators:

>,>=,<,<=

Eg 1:

```
1)  a=10
2)  b=20
3)  print("a > b is ",a>b)
4)  print("a >= b is ",a>=b)
5)  print("a < b is ",a<b)
6)  print("a <= b is ",a<=b)
7)
8)  a > b is False
9)  a >= b is False
10) a < b is True
11) a <= b is True
```

We can apply relational operators for str types

also Eg 2:

```
1)  a="durga"
2)  b="durga"
3)  print("a > b is ",a>b)
4)  print("a >= b is ",a>=b)
```

```
5) print("a < b is ",a<b)
```

```
print("a <= b is ",a<=b)

a > b is False
a >= b is True
a < b is False
a <= b is True
```

Eg:

```
print(True>True) False
print(True>=True) True
print(10 >True) True
print(False > True) False

print(10>'durga')
                TypeError: '>' not supported between instances of 'int' and 'str'
```

Eg:

```
a=10
b=20
if(a>b):
    print("a is greater than b")
else:
    print("a is not greater than b")
```

Outputa is not greater than b

Note: Chaining of relational operators is possible. In the chaining, if all comparisons returns True then only result is True. If atleast one comparison returns False then the result is False

Eg:

```
10<20 ==>True
10<20<30 ==>True
10<20<30<40 ==>True
10<20<30<40>50 ==>False
```

equality operators:

== , !=

We can apply these operators for any type even for incompatible types also

```
>>> 10==20
False
>>> 10!= 20
```

```
True
>>> 10==True
False
>>> False==False
True
>>> "durga"=="durga"
True
>>> 10=="durga"
False
```

Note: Chaining concept is applicable for equality operators. If atleast one comparison returns False then the result is False. otherwise the result is True.

Eg:

```
>>> 10==20==30==40
False
>>> 10==10==10==10
True
```

Logical Operators:

and, or ,not

We can apply for all types.

For boolean types behaviour:

and ==>If both arguments are True then only result is True or ====>If atleast one arugemnt is True then result is True not ==>complement

True and False
==>False True or False
===>True not False
==>True

For non-boolean types behaviour:

0 means False

non-zero means True

empty string is always treated as False

x and y:

==>if x is evaluates to false return x otherwise return y

Eg:

10 and 20

0 and 20

If first argument is zero then result is zero otherwise result is y

x or y:

If x evaluates to True then result is x otherwise result

is y 10 or 20 ==> 10

0 or 20 ==> 20

not x:

If x is evalutates to False then result is True otherwise

False not 10 ==>False

not 0 ==>True

Eg:

```
1)  "durga" and "durgaalwar" ==>durgaalwar
2)  "" and "durga" ==>""
3)  "durga" and "" ==>""
4)  "" or "durga" ==>"durga"
5)  "durga" or ""==>"durga"
6)  not ""==>True
7)  not "durga" ==>False
```

Bitwise Operators:

We can apply these operators bitwise.

These operators are applicable only for int and boolean types.

By mistake if we are trying to apply for any other type then we will get Error.

&,|,^,~,<<,>>

print(4&5) ==>valid
print(10.5 & 5.6) ==>

TypeError: unsupported operand type(s) for &: 'float' and 'float'

print(True & True) ==>valid

& ==> If both bits are 1 then only result is 1 otherwise result is 0

| ==> If atleast one bit is 1 then result is 1 otherwise result is 0

^ ==>If bits are different then only result is 1 otherwise result is 0

~ ==>bitwise complement operator
1==>0 & 0==>1

<< ==>Bitwise Left shift

>> ==>Bitwise Right Shift

print(4&5) ==>4

print(4|5) ==>5

print(4^5) ==>1

Operator	Description
&	If both bits are 1 then only result is 1 otherwise result is 0
|	If atleast one bit is 1 then result is 1 otherwise result is 0
^	If bits are different then only result is 1 otherwise result is 0
~	bitwise complement operator i.e 1 means 0 and 0 means 1
>>	Bitwise Left shift Operator
<<	Bitwise Right shift Operator

bitwise complement operator(~):

We have to apply complement for total

bits. Eg: print(~5) ==>-6

Note:

The most significant bit acts as sign bit. 0 value represents +ve number where as 1 represents -ve value.

positive numbers will be repesented directly in the memory where as -ve numbers will be represented indirectly in 2's complement form.

Shift Operators:

<< Left shift operator

After shifting the empty cells we have to fill with zero

print(10<<2)==>40

0 0 0 0 1 0 1 0

0 0 1 0 1 0 0 0

>> Right Shift operator

After shifting the empty cells we have to fill with sign bit.(0 for +ve and 1 for -ve)

print(10>>2) ==>2

0 0 0 0 1 0 ~~1~~ ~~0~~

0 0 0 0 0 0 1 0

We can apply bitwise operators for boolean types also

print(True & False) ==>False

print(True | False)
===>True print(True ^
False) ==>True print(~True)
==>-2

print(True<<2) ==>4

print(True>>2) ==>0

Assignment Operators:

We can use assignment operator to assign value to the

variable. Eg:

x=10

We can combine asignment operator with some other operator to form compound assignment operator.

Eg: x+=10 ====> x = x+10

The following is the list of all possible compound assignment operators in Python

+=

-=

*=

/=

%=

//=

**=

&=

|=

^=

>>=

<<=

Eg:

```
1)  x=10
2)  x+=20
3)  print(x) ==>30
```

Eg:

```
1)  x=10
2)  x&=5
3)  print(x) ==>0
```

Ternary Operator:

Syntax:

x = firstValue if condition else secondValue

If condition is True then firstValue will be considered else secondValue will be considered.

Eg 1:

```
1)  a,b=10,20
2)  x=30 if a<b else 40
3)  print(x)  #30
```

Eg 2: Read two numbers from the keyboard and print minimum value

```
1)  a=int(input("Enter First Number:"))
2)  b=int(input("Enter Second Number:"))
3)  min=a if a<b else b
4)  print("Minimum Value:",min)
```

Output:

Enter First Number:10
Enter Second Number:30
Minimum Value: 10

Note: Nesting of ternary operator is possible.

Q. Program for minimum of 3 numbers

```
1)  a=int(input("Enter First Number:"))
2)  b=int(input("Enter Second Number:"))
3)  c=int(input("Enter Third Number:"))
4)  min=a if a<b and a<c else b if b<c else c
5)  print("Minimum Value:",min)
```

Q. Program for maximum of 3 numbers

```
1)  a=int(input("Enter First Number:"))
2)  b=int(input("Enter Second Number:"))
3)  c=int(input("Enter Third Number:"))
4)  max=a if a>b and a>c else b if b>c else c
5)  print("Maximum Value:",max)
```

Eg:

```
1)  a=int(input("Enter First Number:"))
2)  b=int(input("Enter Second Number:"))
3) print("Both numbers are equal" if a==b else "First Number is Less than Second Number" if
     a<b else "First Number Greater than Second Number")
```

Output: D:
\python_classes>py test.py
Enter First Number:10
Enter Second Number:10
Both numbers are equal

D:\python_classes>py
test.py Enter First Number:10
Enter Second Number:20
First Number is Less than Second Number

D:\python_classes>py
test.py Enter First Number:20
Enter Second Number:10
First Number Greater than Second Number

Special operators:

Python defines the following 2 special operators

1. Identity Operators
2. Membership operators

1. Identity Operators

We can use identity operators for address comparison. 2 identity operators are available

1. is
2. is not

r1 is r2 returns True if both r1 and r2 are pointing to the same object

r1 is not r2 returns True if both r1 and r2 are not pointing to the same

object Eg:

1)a=10

```
2) b=10
3) print(a is b)        True
4) x=True
5) y=True
6) print( x is y)       True
```

Eg:

```
1  a="durga"
)

2  b="durga"
)

3  print(id(a))
)

4  print(id(b))
)

5  print(a is b)
)
```

Eg:

```
list1=["one","two","three"]
)

list2=["one","two","three"]
)

print(id(list1))
)

print(id(list2))
)

print(list1 is list2) False
)

print(list1 is not list2) True
)

print(list1 == list2)  True
```

Note:

We can use is operator for address comparison where as == operator for content comparison.

2. Membership operators:

We can use Membership operators to check whether the given object present in the given collection.(It may be String,List,Set,Tuple or Dict)

in Returns True if the given object present in the specified Collection

not in Retruns True if the given object not present in the specified Collection

Eg:

```
x="hello learning Python is very easy!!!"
print('h' in x)  True
print('d' in x)      False
print('d' not in x)      True
print('Python' in x)  True
```

Eg:

```
list1=["sunny","bunny","chinny","pinny"]
print("sunny" in list1) True
print("tunny" in list1) False
print("tunny" not in list1) True
```

Operator Precedence:

If multiple operators present then which operator will be evaluated first is decided by operator precedence.

Eg:

print(3+10*2) 23

print((3+10)*2) 26

The following list describes operator precedence in

Python () Parenthesis

** exponential operator

~,- Bitwise complement operator,unary minus operator

*,/,%,// multiplication,division,modulo,floor division

+,- addition,subtraction

<<,>> Left and Right Shift

& bitwise And

^ Bitwise X-OR

| Bitwise OR

>,>=,<,<=, ==, != ==>Relational or Comparison operators

=,+=,-=,*=... ==>Assignment
operators is , is not Identity
Operators

in , not in Membership operators
not Logical not

and Logical and
or Logical or

Eg:

```
1) a=30
2) b=20
3) c=10
4) d=5
5) print((a+b)*c/d)       100.0
6) print((a+b)*(c/        100.0
d))
7) print(a+(b*c)/d)       70.0
8)
9)
10) 3/2*4+3+(10/5)**3-2
11) 3/2*4+3+2.0**3-2
12) 3/2*4+3+8.0-2
13) 1.5*4+3+8.0-2
14) 6.0+3+8.0-2
15) 15.0
```

Mathematical Functions (math Module)

A Module is collection of functions, variables and classes etc.

math is a module that contains several functions to perform mathematical

operations If we want to use any module in Python, first we have to import that

module.

import math

Once we import a module then we can call any function of that

module. import math

print(math.sqrt(16))

print(math.pi)

4.0

3.141592653589793

We can create alias name by using as

keyword. import math as m

Once we create alias name, by using that we can access functions and variables of that module

```
import math as m
print(m.sqrt(16))
print(m.pi)
```

We can import a particular member of a module explicitly as

follows from math import sqrt

from math import sqrt,pi

If we import a member explicitly then it is not required to use module name while accessing.

```
from math import
sqrt,pi print(sqrt(16))
```

```
print(pi)
print(math.pi)    NameError: name 'math' is not defined
```

important functions of math module:

ceil(x)
floor(x)
pow(x,y)
factorial(x)
trunc(x)
gcd(x,y)
sin(x)

cos(x)

tan(x)

....

important variables of math module:

pi3.14
e===>2.71

inf ==>infinity

nan ==>not a number

Q. Write a Python program to find area of circle

pi*r**2

from math import pi
r=16

print("Area of Circle is :",pi*r**2)

OutputArea of Circle is :

804.247719318987

Input And Output Statements

Reading dynamic input from the keyboard:

In Python 2 the following 2 functions are available to read dynamic input from the keyboard.

1.raw_input()
2.input()

1. raw_input():

This function always reads the data from the keyboard in the form of String Format. We have to convert that string type to our required type by using the corresponding type casting methods.

Eg:

```
x=raw_input("Enter First Number:")
print(type(x))          It will always print str type only for any input type
```

2. input():

input() function can be used to read data directly in our required format.We are not required to perform type casting.

```
x=input("Enter
Value) type(x)
```

```
10 ===> int
"durga"===>str
10.5===>float
True==>bool
```

***Note: But in Python 3 we have only input() method and raw_input() method is not available.

Python3 input() function behaviour exactly same as raw_input() method of Python2. i.e every input value is treated as str type only.

raw_input() function of Python 2 is renamed as input() function in Python3

Eg:

```
>>> type(input("Enter value:"))
Enter value:10
<class 'str'>
<class 'str'>
Enter value:10.5
Enter value:True
```

Q. Write a program to read 2 numbers from the keyboard and print sum.

```
x=input("Enter First Number:")
y=input("Enter Second Number:")
j = int(y)
print("The Sum:",i+j)
Enter Second Number:200
Enter First Number:100
```

```
x=int(input("Enter First Number:"))
y=int(input("Enter Second Number:"))
```

```
print("The Sum:",int(input("Enter First Number:"))+int(input("Enter Second Number:")))
```

Q. Write a program to read Employee data from the keyboard and print that data.

```
eno=int(input("Enter Employee No:"))
ename=input("Enter Employee Name:")
esal=float(input("Enter Employee Salary:"))
eaddr=input("Enter Employee Address:")
married=bool(input("Employee Married ?[True|False]:"))
print("Please Confirm Information")
print("Employee No :",eno)
print("Employee Name :",ename)
print("Employee Salary :",esal)
print("Employee Address :",eaddr)
print("Employee Married ? :",married)
```

```
D:\Python_classes>py test.py
Enter Employee No:100
Enter Employee Name:Sunny
Enter Employee Salary:1000
Enter Employee Address:Mumbai
Employee Married ?[True|False]:True
Please Confirm Information
Employee No : 100
Employee Name : Sunny
Employee Salary : 1000.0
Employee Address : Mumbai
Employee Married ? : True
```

How to read multiple values from the keyboard in a single line:

```
a,b= [int(x) for x in input("Enter 2 numbers :").split()]
print("Product is :", a*b)

D:\Python_classes>py test.py
Enter 2 numbers :10 20
 Product is : 200
```

Note: split() function can take space as seperator by default .But we can pass anything as seperator.

Q. Write a program to read 3 float numbers from the keyboard with , seperator and print their sum.

```
a,b,c= [float(x) for x in input("Enter 3 float numbers :").split(',')]
print("The Sum is :", a+b+c)

D:\Python_classes>py test.py
Enter 3 float numbers :10.5,20.6,20.1
The Sum is : 51.2
```

eval():

eval Function take a String and evaluate the

Result. Eg: x = eval("10+20+30")

print(x)

Output: 60

Eg: x = eval(input("Enter Expression")) Enter Expression: 10+2*3/4 Output11.5

eval() can evaluate the Input to list, tuple, set, etc based the provided

Input. Eg: Write a Program to accept list from the keynboard on the

display

```
1) l = eval(input("Enter List"))
2) print (type(l))
3) print(l)
```

```
4)
5) D:\Python_classes>py test.py 10 20
6) 1020
7) 30
```

Note3: If we are trying to access command line arguments with out of range index then we will get Error.

Eg:

```
1)  from sys import argv
2) print(argv[100])
3)
4) D:\Python_classes>py test.py 10 20
5)                 IndexError: list index out of range
```

Note:

In Python there is argparse module to parse command line arguments and display some help messages whenever end user enters wrong input.

input()
raw_input()

command line arguments

output statements:

We can use print() function to display output.

Form-1: print() without any argument

Just it prints new line character

Form-2:

```
1)  print(String):
2)  print("Hello World")
3)  We can use escape characters also
4)  print("Hello \n World")
5)  print("Hello\tWorld")
6)  We can use repetetion operator (*) in the string
7)  print(10*"Hello")
8)  print("Hello"*10)
9)  We can use + operator also
10) print("Hello"+"World")
```

Note:

If both arguments are String type then + operator acts as concatenation operator.

If one argument is string type and second is any other type like int then we will get

Error If both arguments are number type then + operator acts as arithmetic addition

operator. Note:

```
1) print("Hello"+"World")
2) print("Hello","World")
3)
4) HelloWorld
5) Hello World
```

Form-3: print() with variable number of arguments:

```
1. a,b,c=10,20,30
2. print("The Values are :",a,b,c)
4. OutputThe Values are : 10 20 30
```

By default output values are seperated by space.If we want we can specify seperator by using "sep" attribute

```
1. a,b,c=10,20,30
2. print(a,b,c,sep=',')
3. print(a,b,c,sep=':')
4.
6. 10,20,30
```

Form-4:print() with end attribute:

```
1. print("Hello")
2. print("Durga")
```

Output:

```
1. Hello
2. Durga
```

If we want output in the same line with space

```
print("Hello",end=' ')
print("Durga",end=' ')
print("Alwar")
```

Output: Hello Durga Alwar

Note: The default value for end attribute is \n,which is nothing but new line character.

Form-5: print(object) statement:

We can pass any object (like list,tuple,set etc)as argument to the print()

statement. Eg:

```
l=[10,20,30,40]
t=(10,20,30,40)
print(l)
print(t)
```

Form-6: print(String,variable list):

We can use print() statement with String and any number of

arguments. Eg:

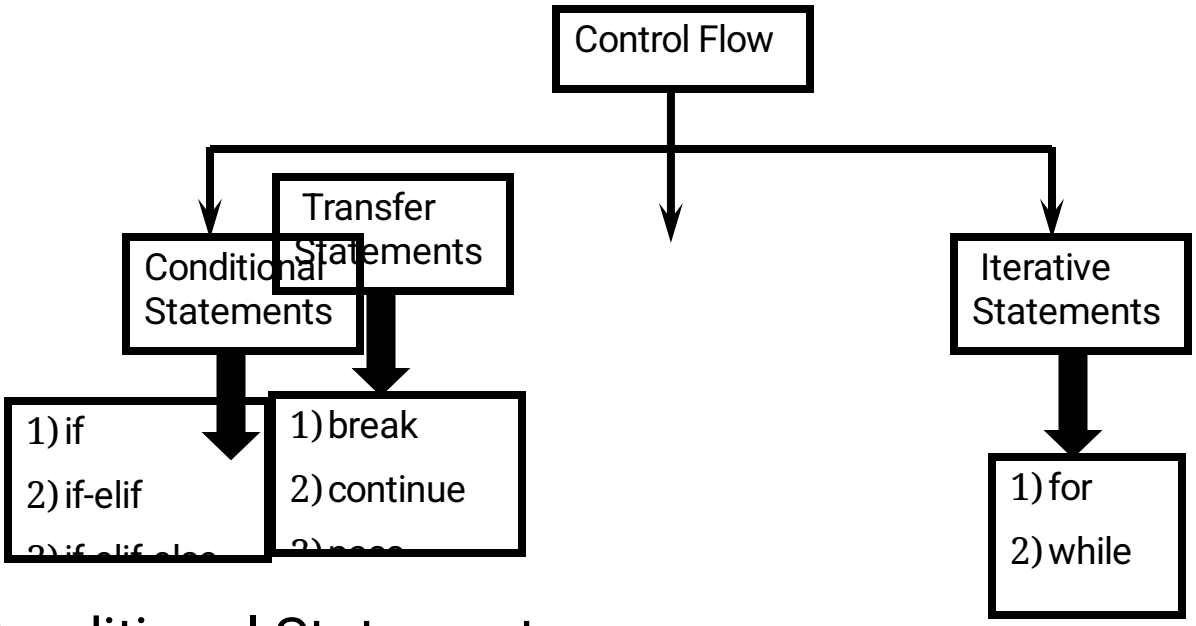
```
s="Nitin"
a=48
s1="java"
s2="Python"
print("Hello",s,"Your Age is",a)
print("You are teaching",s1,"and",s2)
```

Output:

```
1)  Hello Nitin Your Age is 48
2) You are teaching java and Python
```

Flow Control

Flow control describes the order in which statements will be executed at runtime.



I.Conditional Statements

1) if

if condition :

statement or

if condition :
statement-1

statement-2

statement-3

If condition is true then statements will be executed.

Eg:

```
1)  name=input("Enter Name:")
2)  if name=="durga" :
3)                      print("Hello Durga Good Morning")
4)  print("How are you!!!")
5)
6)  D:\Python_classes>py test.py
7)  Enter Name:durga
8)  Hello Durga Good Morning
9)  How are you!!!
10)
11) D:\Python_classes>py test.py
12) Enter Name:Ravi
13) How are you!!!
```

2) if-else:

if condition :

    Action-1
else :

    Action-2

if condition is true then Action-1 will be executed otherwise Action-2 will be

executed. Eg:

```
1)  name=input("Enter Name:")
2)  if name=="durga" :
3)        print("Hello Durga Good Morning")
4)  else:
5)       print("Hello Guest Good Moring")
6)  print("How are you!!!")
7)
8)  D:\Python_classes>py test.py
9)  Enter Name:durga
10) Hello Durga Good Morning
11) How are you!!!
12)
13) D:\Python_classes>py test.py
14) Enter Name:Ravi
15) Hello Guest Good Moring
16) How are you!!!
```

3) if-elif-else:

Syntax:

```
if condition1:
   Action-1
elif condition2:
   Action-2
elif condition3:
   Action-3
 elif condition4:
   Action-4
   ...
else:
   Default Action
```

Based condition the corresponding action will be executed. Eg:

```
1)  brand=input("Enter Your Favourite Brand:")
2)  if brand=="RC" :
3)        print("It is childrens brand")
4)  elif brand=="KF":
5)       print("It is not that much kick")
6)  elif brand=="FO":
7)       print("Buy one get Free One")
8)  else :
9)       print("Other Brands are not recommended")
10)
11)
12) D:\Python_classes>py test.py
13) Enter Your Favourite Brand:RC
14) It is childrens brand
15)
16) D:\Python_classes>py test.py
17) Enter Your Favourite Brand:KF
18) It is not that much kick
19)
20) D:\Python_classes>py test.py
21) Enter Your Favourite Brand:KALYANI
22) Other Brands are not recommended
```

Note:

1. else part is always optional
   Hence the following are various possible syntaxes.

1.if
2.if - else
3. if-elif-else 4.if-elif

2. There is no switch statement in Python

Q. Write a program to find biggest of given 2 numbers from the commad prompt?

```
1)  n1=int(input("Enter First Number:"))
2) n2=int(input("Enter Second Number:"))
3) if n1>n2:
4)     print("Biggest Number is:",n1)
5) else:
6)     print("Biggest Number is:",n2)
7)
8) D:\Python_classes>py test.py
9) Enter First Number:10
10) Enter Second Number:20
11) Biggest Number is: 20
```

Q. Write a program to find biggest of given 3 numbers from the commad prompt?

```
1)  n1=int(input("Enter First Number:"))
2) n2=int(input("Enter Second Number:"))
3) n3=int(input("Enter Third Number:"))
4) if n1>n2 and n1>n3:
5)     print("Biggest Number is:",n1)
6) elif n2>n3:
7)     print("Biggest Number is:",n2)
8) else :
9)     print("Biggest Number is:",n3)
10)
11) D:\Python_classes>py test.py
12) Enter First Number:10
13) Enter Second Number:20
14) Enter Third Number:30
15) Biggest Number is: 30
16)
17) D:\Python_classes>py test.py
18) Enter First Number:10
```

```
19) Enter Second Number:30
20) Enter Third Number:20
21) Biggest Number is: 30
```

Q. Write a program to find smallest of given 2 numbers?

Q. Write a program to find smallest of given 3 numbers?

Q. Write a program to check whether the given number is even or odd?

Q. Write a program to check whether the given number is in between 1 and 100?

```
1)  n=int(input("Enter Number:"))
2)
3) if n>=1 and n<=10:
4)     print("The number",n,"is in between 1 to 10")
5) else:
6)     print("The number",n,"is not in between 1 to 10")
```

Q. Write a program to take a single digit number from the key board and print is value in English word?

```
1)  0==>ZERO
2) 1 ==>ONE
3)
4) n=int(input("Enter a digit from o to 9:"))
5) if n==0 :
6)     print("ZERO")
7) elif n==1:
8)     print("ONE")
9) elif n==2:
10)    print("TWO")
11) elif n==3:
12)    print("THREE")
13) elif n==4:
14)    print("FOUR")
15) elif n==5:
16)    print("FIVE")
17) elif n==6:
18)    print("SIX")
19) elif n==7:
20)    print("SEVEN")
21) elif n==8:
22)    print("EIGHT")
23) elif n==9:
24)    print("NINE")
25) else:
26)    print("PLEASE ENTER A DIGIT FROM 0 TO 9")
```

II.Iterative Statements

If we want to execute a group of statements multiple times then we should go for Iterative statements.

Python supports 2 types of iterative statements.

1. for loop
2. while loop

1) for loop:

If we want to execute some action for every element present in some sequence(it may be string or collection)then we should go for for loop.

Syntax:

```
for x in sequence :
   body
```

where sequence can be string or any collection.

Body will be executed for every element present in the

sequence. Eg 1: To print characters present in the given string

```
1)  s="Sunny Leone"
2) for x in s :
3)      print(x)
4)
5) Output
6) S
7) u
8) n
9) n
10) y
11)
12) L
13) e
14) o
15) n
16) e
```

Eg 2: To print characters present in string index wise:

```
1)  s=input("Enter some String: ")
2) i=0
3) for x in s :
4)      print("The character present at ",i,"index is :",x)
5)      i=i+1
6)
7)
8)    D:\Python_classes>py test.py
9) Enter some String: Sunny Leone
10) The character present at 0 index is : S
11) The character present at 1 index is : u
12) The character present at 2 index is : n
13) The character present at 3 index is : n
14) The character present at 4 index is : y
15) The character present at 5 index is :
16) The character present at 6 index is : L
17) The character present at 7 index is : e
18) The character present at 8 index is : o
19) The character present at 9 index is : n
20) The character present at 10 index is : e
```

Eg 3: To print Hello 10 times

```
1)  for x in range(10) :
2)      print("Hello")
```

Eg 4: To display numbers from 0 to 10

```
1)      for x in
2)      print(x)
```

Eg 5: To display odd numbers from 0 to 20

```
1)  for x in range(21) :
2)
3)      if (x%2!=0)print(x)
```

Eg 6: To display numbers from 10 to 1 in descending order

```
1)        for x in
2)          print(x)
```

Eg 7: To print sum of numbers presenst inside list

```
1)          list=eval(input("Enter
2)     sum=0;
3)     for x in list:
4)        sum=sum+
5)     print("The Sum=",sum)
6)
8) Enter List:[10,20,30,40]
test.py
10)
12) Enter List:[45,67]
```

2) while loop:

If we want to execute a group of statements iteratively until some condition false,then we should go for while loop.

Syntax:

```
while condition :
   body
```

Eg: To print numbers from 1 to 10 by using while loop

```
1)       x=
2)       while x
3)      print(x)
4)      x=x+
```

Eg: To display the sum of first n numbers

```
1)  n=int(input("Enter number:"))
2)  sum=0
3)  i=1
4)  while i<=n:  sum=sum+i
5)      i=i+1   print("The sum of first",n,"numbers is :",sum)
6)
```

Eg: write a program to prompt user to enter some name until entering Durga

```
1) name=""
2) while name!="durga":name=input("Enter Name:")
3)
4) print("Thanks for confirmation")
```

Infinite Loops:

```
1) i=0;
2) while True : i=i+1;
3)
4)     print("Hello",i)
```

Nested Loops:

Sometimes we can take a loop inside another loop,which are also known as nested loops. Eg:

```
1) for i in range(4):
2)     for j in range(4):
3)         print("i=",i," j=",j)
4)
5) Output
6) D:\Python_classes>py test.py
7) i= 0    j= 0
8) i= 0    j= 1
9) i= 0    j= 2
10) i= 0   j= 3
11) i= 1   j= 0
12) i= 1   j= 1
13) i= 1   j= 2
14) i= 1   j= 3
15) i= 2   j= 0
16) i= 2   j= 1
17) i= 2   j= 2
18) i= 2   j= 3
19) i= 3   j= 0
20) i= 3   j= 1
21) i= 3   j= 2
22) i= 3   j= 3
```

Q. Write a program to dispaly *'s in Right angled triangled form

```
1) *
2) * *

4) * * * *

6) * * * * * *


8)
7) * * * * * * *
10) for i in range(1,n+1):

9) n = int(input("Enter number of rows:"))
12     print("* ",end="")
```

Alternative way:

```
1)  n = int(input("Enter number of rows:"))

3) for i in range(1,n+1): print("*" * i)
```

Q. Write a program to display *'s in pyramid style(also known as equivalent triangle)

```
1)                 *
2)       * *
3)      * * *
4)     * * * *
5)    * * * * *
6)   * * * * * *
7) * * * * * * *
8)
9) n = int(input("Enter number of rows:"))
10) for i in range(1,n+1):
11)     print(" " * (n-i),end="")
12)     print("* "*i)
```

III. Transfer Statements

1) break:

We can use break statement inside loops to break loop execution based on some condition.

Eg:

```
or i in range(10):
2)      if i==7:
3)          print("processing is enough..plz
4)          break
5)      print(i)
6)
7) D:\Python_classes>py test.py
8) 0

10) 2

12) 4

14) 6
13) 5
```

Eg:

```
1)   cart=[10,20,600,60,7
2) for item in cart:

4)          print("To place this order insurence must be
5)          break
6)      print(item)
7)
8) D:\Python_classes>py test.py
9) 10
10) 20
11) To place this order insurence must be
```

2) continue:

We can use continue statement to skip current iteration and continue next iteration. Eg 1: To print odd numbers in the range 0 to 9

```
1) for i in range(10):
2)     if i%2==0:
3)                 continue
4)     print(i)
5)
6) D:\Python_classes>py test.py
7) 1
8) 3
9) 5
10) 7
11) 9
```

Eg 2:

```
1)  cart=[10,20,500,700,50,6
2) for item in cart:
4)         print("We cannot process this
5)         continu
6)      print(item)
7)
8) Output
9) D:\Python_classes>py test.py
10) 10
12) We cannot process this item : 500
14) 50
700
```

Eg 3:

```
1)  numbers=[10,20,0,5,0,3
2) for n in numbers:
4)         print("Hey how we can divide with zero..just
5)         continu
6)      print("100/{} = {}".format(n,100/n))
7)
```

```
8) Output
9)
10) 100/10 = 10.0
11) 100/20 = 5.0
12) Hey how we can divide with zero..just skipping
13) 100/5 = 20.0
14) Hey how we can divide with zero..just skipping
15) 100/30 = 3.3333333333333335
```

loops with else block:

Inside loop execution,if break statement not executed ,then only else part will be executed.

else means loop without break

Eg:

```
1)  cart=[10,20,30,40,50]
2) for item in cart:
3)     if item>=500:
4)         print("We cannot process this order")
5)         break
6)     print(item)
7) else:
8)     print("Congrats ...all items processed successfully")
9)
10) Output
11) 10
12) 20
13) 30
14) 40
15) 50
16) Congrats ...all items processed successfully
```

Eg:

```
1)  cart=[10,20,600,30,40,50]
2) for item in cart:
3)     if item>=500:
4)         print("We cannot process this order")
5)         break
6)     print(item)
7) else:
8)     print("Congrats ...all items processed successfully")
```

```
9)
10) Output
11) D:\Python_classes>py test.py
12) 10
13) 20
14) We cannot process this order
```

Q. What is the difference between for loop and while loop in Python?

We can use loops to repeat code execution

Repeat code for every item in sequence ==>for loop
Repeat code as long as condition is true ==>while loop

Q. How to exit from the loop?
by using break statement

Q. How to skip some iterations inside loop? by using continue statement.

Q. When else part will be executed wrt loops? If loop executed without break

3) pass statement:

pass is a keyword in Python.

In our programming syntactically if block is required which won't do anything then we can define that empty block with pass keyword.

pass

|- It is an empty statement

|- It is null statement

|- It won't do

anything Eg:

if True:

SyntaxError: unexpected EOF while parsing

if True: pass

==>valid

def m1():

SyntaxError: unexpected EOF while parsing

def m1(): pass

use case of pass:

Sometimes in the parent class we have to declare a function with empty body and child class responsible to provide proper implementation. Such type of empty body we can define by using pass keyword. (It is something like abstract method in java)

Eg:

def m1():

pass Eg:

```
1)  for i in range(100):
2)      if i%9==0:
3)          print(i)
4)      else:pass
5)
6) D:\Python_classes>py test.py
7) 0
8) 9
9) 18
10) 27
11) 36
12) 45
13) 54
14) 63
15) 72
16) 81
17) 90
18) 99
```

del statement:

del is a keyword in Python.

After using a variable, it is highly recommended to delete that variable if it is no longer required,so that the corresponding object is eligible for Garbage Collection.

We can delete variable by using del

keyword. Eg:

```
1)  x=10
2) print(x)
```

3) del x

After deleting a variable we cannot access that variable otherwise we will get

NameError. Eg:

```
1)  x=10
2) del x
3) print(x)
```

NameError: name 'x' is not defined.

Note:

We can delete variables which are pointing to immutable objects.But we cannot delete the elements present inside immutable object.

Eg:

```
1)  s="durga"
2) del s
3) print(s) ==>valid

4) del s[0] ==>TypeError: 'str' object doesn't support item deletion
```

Difference between del and None:

In the case del, the variable will be removed and we cannot access that variable(unbind operation)

```
1)  s="durga"
2)  del s      print(s)      ==>NameError: name 's' is not defined.
```

But in the case of None assignment the variable won't be removed but the corresponding object is eligible for Garbage Collection(re bind operation). Hence after assigning with None value,we can access that variable.

```
1)s="durga"
2)   s=None
3)              print(s)    # None
```

String Data Type

The most commonly used object in any project and in any programming language is String only. Hence we should aware complete information about String data type.

What is String?

Any sequence of characters within either single quotes or double quotes is considered as a String.

Syntax:

```
s='durga'
s="durga"
```

Note: In most of other languges like C, C++,Java, a single character with in single quotes is treated as char data type value. But in Python we are not having char data type.Hence it is treated as String only.

Eg:

```
>>> ch='a'
>>> type(ch)
<class 'str'>
```

How to define multi-line String literals:

We can define multi-line String literals by using triple single or double quotes.

Eg:

```
>>> s="NIti
Gandhi
Alwar"
```

We can also use triple quotes to use single quotes or double quotes as symbol inside String literal.

Eg:

```
s='This is ' single quote symbol' ==>invalid
s='This is \' single quote symbol' ==>valid
s="This is ' single quote symbol"====>valid
s='This is " double quotes symbol' ==>valid
s='The "Python Notes" by 'nitin' is very helpful' ==>invalid
s="The "Python Notes" by 'nitin' is very helpful"==>invalid
s='The \"Python Notes\" by \'nitin\' is very helpful' ==>valid
s="'The "Python Notes" by 'nitin' is very helpful'" ==>valid
```

How to access characters of a String:

We can access characters of a string by using the following ways.

1. By using index
2. By using slice operator

1. By using index:

Python supports both +ve and -ve index.
+ve index means left to right(Forward direction)
-ve index means right to left(Backward direction)

Eg:

s='durga'

diagram

Eg:

```
>>> s='durga'
>>> s[0]
'd'
>>> s[4]
'a'
>>> s[-1]
'a'
>>> s[10]
IndexError: string index out of range
```

Note: If we are trying to access characters of a string with out of range index then we will get error saying : IndexError

2. Accessing characters by using slice operator:

Syntax: s[bEginindex:endindex:step]

bEginindex:From where we have to consider slice(substring)
endindex: We have to terminate the slice(substring) at endindex-1
step: incremented value

Note: If we are not specifying bEgin index then it will consider from bEginning of the string.
If we are not specifying end index then it will consider up to end of the string
The default value for step is 1

Eg:

```
1)  >>> s="Learning Python is very very easy!!!"
2)  >>> s[1:7:1]
3)  'earnin'
4)  >>> s[1:7]
5)  'earnin'
6)  >>> s[1:7:2]
7)  'eri'
8)  >>> s[:7]
9)  'Learnin'
10) >>> s[7:]
11) 'g Python is very very easy!!!'
12) >>> s[::]
13) 'Learning Python is very very easy!!!'
14) >>> s[:]
15) 'Learning Python is very very easy!!!'
16) >>> s[::-1]
17) '!!!ysae yrev yrev si nohtyP gninraeL'
```

Behaviour of slice operator:

s[bEgin:end:step]

step value can be either +ve or –ve

if +ve then it should be forward direction(left to right) and we have to consider bEgin to end-1 if

-ve then it should be backward direction(right to left) and we have to consider bEgin to end+1

***Note:

In the backward direction if end value is -1 then result is always empty. In the forward direction if end value is 0 then result is always empty.

In forward direction:

default value for bEgin: 0
default value for end: length of string
default value for step: +1

In backward direction:

default value for bEgin: -1
default value for end: -(length of string+1)

Note: Either forward or backward direction, we can take both +ve and -ve values for bEgin and end index.

Mathematical Operators for String:

We can apply the following mathematical operators for Strings.

1. + operator for concatenation
2. * operator for repetition

```
print("durga"+"alwar")
#durgaalwar

print("durga"*2) #durgadurga
```

Note:

1. To use + operator for Strings, compulsory both arguments should be str type

2. To use * operator for Strings, compulsory one argument should be str and other argument should be int

len() in-built function:

We can use len() function to find the number of characters present in the string.

Eg:

```
s='durga' print(
len(s)) #5
```

Q. Write a program to access each character of string in forward and backward direction by using while loop?

```
1)  s="Learning Python is very easy !!!"
2)  n=len(s)
3)  i=0
4)  print("Forward direction")
5)  while i<n:
6)      print(s[i],end=' ')
7)      i +=1
8)  print("Backward direction")
9)  i=-1
10) while i>=-n:
11)     print(s[i],end=' ')
12)     i=i-1
```

Alternative ways:

```
1)  s="Learning Python is very easy !!!"
2)  print("Forward direction")
3)  for i in s:
4)      print(i,end=' ')
5)
6)  print("Forward direction")
7)  for i in s[::]:
8)      print(i,end=' ')
9)
10) print("Backward direction")
11) for i in s[::-1]:
12)     print(i,end=' ')
```

Checking Membership:

We can check whether the character or string is the member of another string or not by using in and not in operators

```
s='durga'
print('d' in s) #True
print('z' in s) #False
```

Program:

```
1)  s=input("Enter main string:")
2)  subs=input("Enter sub string:")
3)  if subs in s:
4)      print(subs,"is found in main string")
5)  else:
```

```
6)      print(subs,"is not found in main string")
```

Output:

```
D:\python_classes>py test.py
Enter main string:nitin Gandhi alwar
 Enter sub string:nitin
nitin is found in main string

D:\python_classes>py test.py
Enter main string:nitin Gandhi alwar
Enter sub string:python
python is not found in main string
```

Comparison of Strings:

We can use comparison operators (<,<=,>,>=) and equality operators(==,!=) for strings.

Comparison will be performed based on alphabetical order.

Eg:

```
1)  s1=input("Enter first string:")
2)  s2=input("Enter Second string:")
3)  if s1==s2:
4)      print("Both strings are equal")
5)  elif s1<s2:
6)      print("First String is less than Second String")
7)  else:
8)      print("First String is greater than Second String")
```

Output: D:
\python_classes>py test.py
Enter first string:durga
Enter Second string:durga
Both strings are equal

D:\python_classes>py
test.py Enter first
string:durga
Enter Second string:ravi
First String is less than Second String

D:\python_classes>py
test.py Enter first
string:durga
Enter Second string:anil
First String is greater than Second String

Removing spaces from the string:

We can use the following 3 methods

1. rstrip()===>To remove spaces at right hand side
2. lstrip()===>To remove spaces at left hand side
3. strip() ==>To remove spaces both sides

Eg:

```
1)  city=input("Enter your city Name:")
2)  scity=city.strip()
3)  if scity=='Hyderabad':
4)      print("Hello Hyderbadi..Adab")
5)  elif scity=='Chennai':
6)      print("Hello Madrasi...Vanakkam")
7)  elif scity=="Bangalore":
8)     print("Hello Kannadiga...Shubhodaya")
9)  else:
10)     print("your entered city is invalid")
```

Finding Substrings:

We can use the following 4 methods

For forward direction:

find()
index()

For backward direction:

rfind()
rindex()

1. find():

s.find(substring)

Returns index of first occurrence of the given substring. If it is not available then we will get -1

Eg:

```
1)  s="Learning Python is very easy"
```

```
print(s.find("Python")) #9
print(s.find("Java")) # -1
print(s.find("r"))#3
print(s.rfind("r"))#21
```

Note: By default find() method can search total string. We can also specify the boundaries to search.

```
s.find(substring,bEgin,end)
```

It will always search from bEgin index to end-1 index

Eg:

```
s="durgaravipavanshiva"
print(s.find('a'))#4
print(s.find('a',7,15))#10
print(s.find('z',7,15))#-1
```

index() method:

index() method is exactly same as find() method except that if the specified substring is not available then we will get ValueError.

Eg:

```
s=input("Enter main string:")
subs=input("Enter sub string:")
try:
    n=s.index(subs)
except ValueError:
    print("substring not found")
else:
    print("substring found")
```

Output:

```
D:\python_classes>py test.py
Enter main string:learning python is very easy
Enter sub string:python
substring found

D:\python_classes>py test.py
Enter main string:learning python is very easy
Enter sub string:java
substring not found
```

Q. Program to display all positions of substring in a given main string

```
1)  s=input("Enter main string:")
2)  subs=input("Enter sub string:")
3)  flag=False
4)  pos=-1
5)  n=len(s)
6)  while True:
7)                  pos=s.find(subs,pos+1,n)
8)      if pos==-1:
9)                      break
10)     print("Found at position",pos)
11)                 flag=True
12) if flag==False:
13)                 print("Not Found")
```

Output:

```
D:\python_classes>py test.py
Enter main
string:abbababababacdefg Enter
sub string:a
Found at position 0
Found at position 3
Found at position 5
Found at position 7
Found at position 9
Found at position
11

D:\python_classes>py test.py
Enter main string:abbababababacdefg
Enter sub string:bb
Found at position 1
```

Counting substring in the given String:

We can find the number of occurrences of substring present in the given string by using count() method.

1. s.count(substring) ==> It will search through out the string
2. s.count(substring, bEgin, end) ===> It will search from bEgin index to end-1 index

Eg:

```
1)  s="abcabcabcabcadda"
2)  print(s.count('a'))
3)  print(s.count('ab'))
4)  print(s.count('a',3,7))
```

Output:

6
4
2

Replacing a string with another string:

s.replace(oldstring,newstring)

inside s, every occurrence of oldstring will be replaced with newstring.

Eg1:

```
s="Learning Python is very difficult"
s1=s.replace("difficult","easy")
print(s1)
```

Output:

Learning Python is very easy

Eg2: All occurrences will be replaced

```
s="abababababab
" s1=s.replace("a","b")
print(s1)
```

Output: bbbbbbbbbbbbb

Q. String objects are immutable then how we can change the content by using replace() method.

Once we creates string object, we cannot change the content.This non changeable behaviour is nothing but immutability. If we are trying to change the content by using any method, then with those changes a new object will be created and changes won't be happend in existing object.

Hence with replace() method also a new object got created but existing object won't be changed.

Eg:

```
s="abab" s1=s.replace("a","b")
print(s,"is available at :",id(s))
print(s1,"is available at :",id(s1))
```

Output:

abab is available at : 4568672
bbbb is available at : 4568704

In the above example, original object is available and we can see new object which was created because of replace() method.

Splitting of Strings:

We can split the given string according to specified seperator by using split() method.

l=s.split(seperator)

The default seperator is space. The return type of split() method is List

Eg1:

```
1)  s="nitin Gandhi alwar"
2)  l=s.split()
3)  for x in l:
4)      print(x)
```

Output:
nitin

Gandhi
alwar

Eg2:

```
1)  s="22-02-2018"
2)  l=s.split('-')
3)  for x in l:
4)      print(x)
```

Output:

22
02
2018

Joining of Strings:

We can join a group of strings(list or tuple) wrt the given seperator.

s=seperator.join(group of strings)

Eg:
t=('sunny','bunny','chinny')
s='-'.join(t)
print(s)

Output: sunny-bunny-chinny

Eg2:

```
l=['hyderabad','singapore','london','dubai']
s=':'.join(l)
print(s)
```

hyderabad:singapore:london:dubai

Changing case of a String:

We can change case of a string by using the following 4 methods.

1. upper()===>To convert all characters to upper case
2. lower() ===>To convert all characters to lower case
3. swapcase()===>converts all lower case characters to upper case and all upper case characters to lower case
4. title() ===>To convert all character to title case. i.e first character in every word should be upper case and all remaining characters should be in lower case.
5. capitalize() ==>Only first character will be converted to upper case and all remaining characters can be converted to lower case

Eg:

```
s='learning Python is very Easy'
print(s.upper())
print(s.lower())
print(s.swapcase())
print(s.title())
print(s.capitalize())
```

Output:

```
LEARNING PYTHON IS VERY EASY
learning python is very easy
LEARNING pYTHON IS VERY eASY
Learning Python Is Very Easy
Learning python is very easy
```

Checking starting and ending part of the string:

Python contains the following methods for this purpose

1. s.startswith(substring)
2. s.endswith(substring)

Eg:

```
s='learning Python is very easy'
print(s.startswith('learning'))
print(s.endswith('learning'))
print(s.endswith('easy'))
```

Output:

True
False
True

To check type of characters present in a string:

Python contains the following methods for this purpose.

1) isalnum(): Returns True if all characters are alphanumeric(a to z , A to Z ,0 to9)
2) isalpha(): Returns True if all characters are only alphabet symbols(a to z,A to Z)
3) isdigit(): Returns True if all characters are digits only(0 to 9)
4) islower(): Returns True if all characters are lower case alphabet symbols
5) isupper(): Returns True if all characters are upper case aplhabet symbols
6) istitle(): Returns True if string is in title case
7) isspace(): Returns True if string contains only spaces

Eg:

```
print('Durga786'.isalnum())      #True
print('durga786'.isalpha())      #False
print('durga'.isalpha())         #True
print('durga'.isdigit())         #False
print('786786'.isdigit())        #True
print('abc'.islower())
#True
print('Abc'.islower())
#False print('abc123'.islower())
#True print('ABC'.isupper()) #True
print('Learning python is Easy'.istitle())
#False print('Learning Python Is
Easy'.istitle()) #True print(' '.isspace())
#True
```

Demo Program:

```
1)  s=input("Enter any character:")
2)  if s.isalnum():
3)      print("Alpha Numeric Character")
4)      if s.isalpha():
5)          print("Alphabet character")
6)          if s.islower():
7)              print("Lower case alphabet character")
8)          else:
9)              print("Upper case alphabet character")
10)     else:
11)         print("it is a digit")
12) elif s.isspace():
13)     print("It is space character")
14) else:
```

```
15)                print("Non Space Special Character")
```

```
D:\python_classes>py
test.py Enter any character:7
Alpha Numeric Character
it is a digit

D:\python_classes>py
test.py Enter any character:a
Alpha Numeric Character
Alphabet character
Lower case alphabet character

D:\python_classes>py
test.py Enter any character:$
Non Space Special Character

D:\python_classes>py
test.py Enter any character:A
Alpha Numeric Character
Alphabet character
Upper case alphabet character
```

Formatting the Strings:

We can format the strings with variable values by using replacement operator {} and format() method.

Eg:

```
name='abc'
salary=1000
0 age=48
print("{} 's salary is {} and his age is {}".format(name,salary,age))
print("{0} 's salary is {1} and his age is {2}".format(name,salary,age))
print("{x} 's salary is {y} and his age is
{z}".format(z=age,y=salary,x=name))
```

Output:

```
abc 's salary is 10000 and his age is 48
abc 's salary is 10000 and his age is 48
abc 's salary is 10000 and his age is 48
```

List Data Structure

If we want to represent a group of individual objects as a single entity where insertion order preserved and duplicates are allowed, then we should go for List.

insertion order preserved.
duplicate objects are allowed
heterogeneous objects are
allowed.

List is dynamic because based on our requirement we can increase the size and decrease the size.

In List the elements will be placed within square brackets and with comma seperator.

We can differentiate duplicate elements by using index and we can preserve insertion order by using index. Hence index will play very important role.

Python supports both positive and negative indexes. +ve index means from left to right where as negative index means right to left

[10,"A","B",20, 30, 10]

-6	-5	-4	-3	-2	-1
10	A	B	20	30	10
0	1	2	3	4	5

List objects are mutable.i.e we can change the content.

Creation of List Objects:

1. We can create empty list object as follows...

```
1) list=[]
2) print(list)
3) print(type(list))
4)
5) []
6) <class 'list'>
```

2. If we know elements already then we can create list as follows list=[10,20,30,40]

3. With dynamic input:

```
1) list=eval(input("Enter List:"))
2) print(list)
3) print(type(list))
4)
5)   D:\Python_classes>py test.py
6) Enter List:[10,20,30,40]
7) [10, 20, 30, 40]
8) <class 'list'>
```

4. With list() function:

```
1   l=list(range(0,10,2))
)

2   print(l)
)

3   print(type(l))
)

4
)

5   D:\Python_classes>py test.py
)

6   [0, 2, 4, 6, 8]
)

7   <class 'list'>
)
```

Eg:

```
1   s="durga"
)

2) l=list(s)
3) print(l)
4)
5) D:\Python_classes>py test.py
6) ['d', 'u', 'r', 'g', 'a']
```

5. with split() function:

```
1) s="Learning Python is very very easy !!!"
2) l=s.split()
```

```
print(l)
print(type(l))

D:\Python_classes>py test.py
['Learning', 'Python', 'is', 'very', 'very', 'easy', '!!!']
<class 'list'>
```

Note:

Sometimes we can take list inside another list,such type of lists are called nested lists.
[10,20,[30,40]]

Accessing elements of List:

We can access elements of the list either by using index or by using slice operator(:)

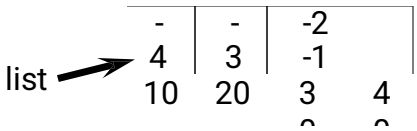
1. By using index:

List follows zero based index. ie index of first element is zero. List supports both +ve and -ve indexes.

+ve index meant for Left to Right

-ve index meant for Right to Left

list=[10,20,30,40]



print(list[0]) ==>10

print(list[-1]) ==>40

print(list[10]) ==>IndexError: list index out of range

2. By using slice operator:

Syntax:

list2= list1[start:stop:step]

start ==>it indicates the index where slice has to start
default value is 0

stop ===>It indicates the index where slice has to end
default value is max allowed index of list ie length of the list

step ==>increment value
default value is 1

Eg:

```
1) n=[1,2,3,4,5,6,7,8,9,10]
2  print(n[2:7:2])
```

```
)

 3) print(n[4::2])
4    print(n[3:7])
)

5    print(n[8:2:-2])
)

6    print(n[4:100])
)
```

```
7)
8)    Output
9)  D:\Python_classes>py test.py
10) [3, 5, 7]
11) [5, 7, 9]
12) [4, 5, 6, 7]
13) [9, 7, 5]
14) [5, 6, 7, 8, 9, 10]
```

List vs mutability:

Once we creates a List object,we can modify its content. Hence List objects are mutable. Eg:

```
1)  n=[10,20,30,40]
2)  print(n)
3)  n[1]=777
4)  print(n)
5)
6)  D:\Python_classes>py test.py
7)  [10, 20, 30, 40]
8)  [10, 777, 30, 40]
```

Traversing the elements of List:

The sequential access of each element in the list is called traversal.

1. By using while loop:

```
1)   n=[0,1,2,3,4,5,6,7,8,9,10]
2)  i=0
3)  while i<len(n):
4)      print(n[i])

6)      i=i+1

8)  0
7)  D:\Python_classes>py test.py
10) 2

12) 4

14) 6

16) 8
```

```
18) 10
```

2. By using for loop:

```
1)  n=[0,1,2,3,4,5,6,7,8,9,10]
2)  for n1 in n:
3)      print(n1)
4)
5)  D:\Python_classes>py test.py
6)  0
7)  1
8)  2
9)  3
10) 4
11) 5
12) 6
13) 7
14) 8
15) 9
16) 10
```

3. To display only even numbers:

```
1)  n=[0,1,2,3,4,5,6,7,8,9,10]
2)  for n1 in n:
3)      if n1%2==0:
4)          print(n1)
5)
6)  D:\Python_classes>py test.py
7)  0
8)  2
9)  4
10) 6
11) 8
12) 10
```

4. To display elements by index wise:

```
1)  l=["A","B","C"]
2)  x=len(l)
3)  for i in range(x):
4)      print(l[i],"is available at positive index: ",i,"and at negative index: ",i-x)
5)
6)  Output
7)  D:\Python_classes>py test.py
8)  A is available at positive index: 0 and at negative index: -3
9)  B is available at positive index: 1 and at negative index: -2
10) C is available at positive index: 2 and at negative index: -1
```

Important functions of List:

I. To get information about list:

1. len():

returns the number of elements present in the list

Eg: n=[10,20,30,40]

print(len(n))==>4

2. count():

It returns the number of occurrences of specified item in the list

```
1) n=[1,2,2,2,2,3,3]
2) print(n.count(1))
3) print(n.count(2))
4) print(n.count(3))
5) print(n.count(4))
6)
7)    Output
8) D:\Python_classes>py test.py
9) 1
10) 4
11) 2
12) 0
```

3. index() function:

returns the index of first occurrence of the specified

item. Eg:

```
1) n=[1,2,2,2,2,3,3]
2) print(n.index(1))     ==>0
3) print(n.index(2))              ==>1
4) print(n.index(3))     ==>5
5) print(n.index(4))              ==>ValueError: 4 is not in list
```

Note: If the specified element not present in the list then we will get ValueError.Hence before index() method we have to check whether item present in the list or not by using in operator.

print(4 in n)==>False

II. Manipulating elements of List:

1. append() function:

We can use append() function to add item at the end of the list. Eg:

```
1)  list=[]
2)  list.append("A")
3)  list.append("B")
4)  list.append("C")
5)  print(list)
6)
7)  D:\Python_classes>py test.py
8)  ['A', 'B', 'C']
```

Eg: To add all elements to list upto 100 which are divisible by 10

```
1)  list=[]
2)  for i in range(101):
3)      if i%10==0:
4)          list.append(i)
5)  print(list)
6)
8)  D:\Python_classes>py test.py
```

2. insert() function:

To insert item at specified index position

```
1)  n=[1,2,3,4,5]
2)  n.insert(1,888)
3)  print(n)
4)
5)  D:\Python_classes>py test.py
6)  [1, 888, 2, 3, 4, 5]
```

Eg:

```
1)  n=[1,2,3,4,5]
2)  n.insert(10,777)
3)  n.insert(-10,999)
4)  print(n)
5)
```

```
6)  D:\Python_classes>py test.py
7) [999, 1, 2, 3, 4, 5, 777]
```

Note: If the specified index is greater than max index then element will be inserted at last position. If the specified index is smaller than min index then element will be inserted at first position.

Differences between append() and insert()

append()	insert()
In List when we add any element it will come in last i.e. it will be last element.	In List we can insert any element in particular index number

3. extend() function:

To add all items of one list to another list

l1.extend(l2)

all items present in l2 will be added to l1

Eg:

```
1  order1=["Chicken","Mutton","Fish"]
)
2  order2=["RC","KF","FO"]
)
3  order1.extend(order2)
)
4  print(order1)
)
5
)
6  D:\Python_classes>py test.py
)
7  ['Chicken', 'Mutton', 'Fish', 'RC', 'KF', 'FO']
)
```

Eg:

```
1) order=["Chicken","Mutton","Fish"]
2) order.extend("Mushroom")
3) print(order)
4)
5)    D:\Python_classes>py test.py
6) ['Chicken', 'Mutton', 'Fish', 'M', 'u', 's', 'h', 'r', 'o', 'o', 'm']
```

4. remove() function:

We can use this function to remove specified item from the list.If the item present multiple times then only first occurrence will be removed.

```
1)  n=[10,20,10,30]
2) n.remove(10)
3) print(n)
4)
5) D:\Python_classes>py test.py
6) [20, 10, 30]
```

If the specified item not present in list then we will get ValueError

```
1)  n=[10,20,10,30]
2) n.remove(40)

3) print(n)
```

Note: Hence before using remove() method first we have to check specified element present in the list or not by using in operator.

5. pop() function:

It removes and returns the last element of the list.

This is only function which manipulates list and returns some

element. Eg:

```
1)  n=[10,20,30,40]
2) print(n.pop())
3) print(n.pop())
4) print(n)
5)
6) D:\Python_classes>py test.py
7) 40
8) 30
9) [10, 20]
```

If the list is empty then pop() function raises

IndexError Eg:

```
1)  n=[]
2) print(n.pop()) ==> IndexError: pop from empty list
```

Note:

1. pop() is the only function which manipulates the list and returns some value
2. In general we can use append() and pop() functions to implement stack datastructure by using list,which follows LIFO(Last In First Out) order.

In general we can use pop() function to remove last element of the list. But we can use to remove elements based on index.

n.pop(index)===>To remove and return element present at specified index.
n.pop()==>To remove and return last element of the list

1)n=[10,20,30,40,50,60]

```
2)  print(n.pop()) #60
3)  print(n.pop(1))    #20

4)  print(n.pop(10)) ==>IndexError: pop index out of range
```

Differences between remove() and pop()

remove()	pop()
1) We can use to remove special element from the List.	1) We can use to remove last element from the List.
2) It can't return any value.	2) It returned removed element.
3) If special element not available then we get VALUE ERROR.	3) If List is empty then we get Error.

Note:

List objects are dynamic. i.e based on our requirement we can increase and decrease the size.

append(),insert() ,extend() ===>for increasing the size/growable nature remove(), pop() =======>for decreasing the size /shrinking nature

III. Ordering elements of List:

1. reverse():

We can use to reverse() order of elements of list.

```
1) n=[10,20,30,40]
2) n.reverse()
3) print(n)
4)
5) D:\Python_classes>py test.py
6) [40, 30, 20, 10]
```

2. sort() function:

In list by default insertion order is preserved. If want to sort the elements of list according to default natural sorting order then we should go for sort() method.

For numbers ==>default natural sorting order is Ascending Order

For Strings ==> default natural sorting order is Alphabetical Order

```
1) n=[20,5,15,10,0]
2) n.sort()
3) print(n)        #[0,5,10,15,20]
4)

6) s.sort()
```

Note: To use sort() function, compulsory list should contain only homogeneous elements. otherwise we will get TypeError

Eg:

```
1) n=[20,10,"A","B"]
2) n.sort()
3) print(n)
4)
```

Note: In Python 2 if List contains both numbers and Strings then sort() function first sort numbers followed by strings

```
1) n=[20,"B",10,"A"]
2) n.sort()
```

```
3) print(n)# [10,20,'A','B']
```

But in Python 3 it is invalid.

To sort in reverse of default natural sorting order:

We can sort according to reverse of default natural sorting order by using reverse=True argument.

Eg:

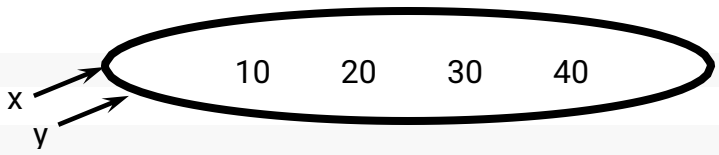
```
1.  n=[40,10,30,20]
2. n.sort()
3. print(n) ==>[10,20,30,40]
4. n.sort(reverse=True)
5. print(n) ===>[40,30,20,10]
6. n.sort(reverse=False)
7. print(n) ==>[10,20,30,40]
```

Aliasing and Cloning of List objects:

The process of giving another reference variable to the existing list is called

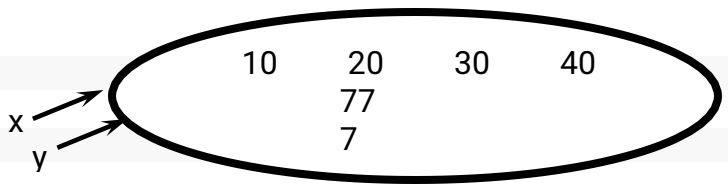
aliasing. Eg:

```
1) x=[10,20,30,40
   ]
2)
   y=x
3) print(id(y))
```



The problem in this approach is by using one reference variable if we are changing content,then those changes will be reflected to the other reference variable.

```
1) x=[10,20,30,40]
2) y=x
3) y[1]=77
```



To overcome this problem we should go for cloning.

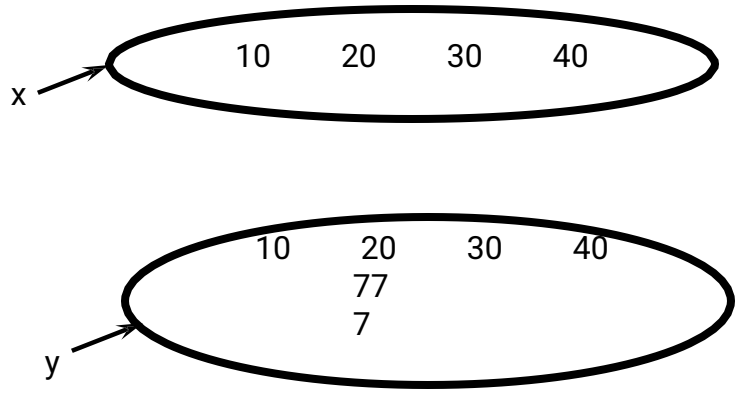
The process of creating exactly duplicate independent object is called

cloning. We can implement cloning by using slice operator or by using

copy() function

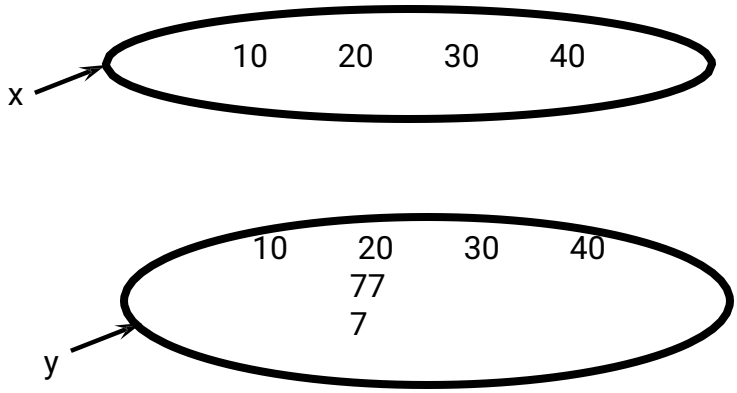
1. By using slice operator:

```
1)  x=[10,20,30,40]
2)  y=x[:]
3)  y[1]=777
4)  print(x) ==>[10,20,30,40]
5)  print(y) ==>[10,777,30,40]
```



2. By using copy() function:

```
1)  x=[10,20,30,40]
2)  y=x.copy()
3)  y[1]=777
4)  print(x) ==>[10,20,30,40]
5)  print(y) ==>[10,777,30,40]
```



Q. Difference between = operator and copy() function

= operator meant for aliasing
copy() function meant for
cloning

Using Mathematical operators for List Objects:

We can use + and * operators for List objects.

1. Concatenation operator(+):

We can use + to concatenate 2 lists into a single list

```
1) a=[10,20,30]
2) b=[40,50,60]
3)
4) print(c) ==>[10,20,30,40,50,60]
```

Note: To use + operator compulsory both arguments should be list objects,otherwise we will get TypeError.

Eg:

c=a+40 ==>TypeError: can only concatenate list (not "int") to list c=a+[40]==>valid

2. Repetition Operator(*):

We can use repetition operator * to repeat elements of list specified number of

times Eg:

```
1) x=[10,20,30]
2) y=x*3
3) print(y)==>[10,20,30,10,20,30,10,20,30]
```

Comparing List objects

We can use comparison operators for List

objects. Eg:

```
1. x=["Dog","Cat","Rat"]
2. y=["Dog","Cat","Rat"]
3. z=["DOG","CAT","RAT"]
4. print(x==y) True
5. print(x==z)    False
6. print(x != z)      True
```

Note:

Whenever we are using comparison operators(==,!=) for List objects then the following should be considered

1.The number of elements
2.The order of elements
3.The content of elements (case sensitive)

Note: When ever we are using relatational operators(<,<=,>,>=) between List objects,only first element comparison will be performed.

Eg:

```
1.  x=[50,20,30]
2. y=[40,50,60,100,200]
3. print(x>y) True
4. print(x>=y) True
5. print(x<y)      False
6. print(x<=y) False
```

Eg:

```
1.  x=["Dog","Cat","Rat"]
2. y=["Rat","Cat","Dog"]
3. print(x>y) False
4. print(x>=y) False
5. print(x<y)  True
6. print(x<=y) True
```

Membership operators:

We can check whether element is a member of the list or not by using memebership operators.

in operator not
in operator

Eg:

```
1.  n=[10,20,30,40]
2. print (10 in n)

4. print (50 in n)

6.
5. print (50 not in n)
```

```
True
False
False
True
```

clear() function:

We can use clear() function to remove all elements of

List. Eg:

```
n=[10,20,30,40]
print(n)
n.clear()
print(n)

Output
D:\Python_classes>py test.py
[10, 20, 30, 40]
[]
```

Nested Lists:

Sometimes we can take one list inside another list. Such type of lists are called nested lists.

Eg:

```
n=[10,20,[30,40]]
print(n)
print(n[0])
print(n[2])
print(n[2][0])
print(n[2][1])

Output
D:\Python_classes>py test.py
[10, 20, [30, 40]]
10
[30, 40]
30
40
```

Note: We can access nested list elements by using index just like accessing multi dimensional array elements.

Nested List as Matrix:

In Python we can represent matrix by using nested lists.

```
1)  n=[[10,20,30],[40,50,60],[70,80,90]]
2)  print(n)
3)  print("Elements by Row wise:")
4)  for r in n:
5)      print(r)
6)  print("Elements by Matrix style:")
7)  for i in range(len(n)):
8)      for j in range(len(n[i])):
9)          print(n[i][j],end=' ')
10)     print()
11)
12) Output
13) D:\Python_classes>py test.py
14) [[10, 20, 30], [40, 50, 60], [70, 80, 90]]
15) Elements by Row wise:
16) [10, 20, 30]
17) [40, 50, 60]
18) [70, 80, 90]
19) Elements by Matrix style:
20) 10 20 30
21) 40 50 60
22) 70 80 90
```

List Comprehensions:

It is very easy and compact way of creating list objects from any iterable objects(like list,tuple,dictionary,range etc) based on some condition.

Syntax:

list=[expression for item in list if

condition] Eg:

```
1)  s=[ x*x for x in range(1,11)]
2)  print(s)
3)  v=[2**x for x in range(1,6)]
4)  print(v)
5)  m=[x for x in s if x%2==0]
6)  print(m)
7)
8)  Output
9)  D:\Python_classes>py test.py
10) [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```

```
[2, 4, 8, 16, 32]
```

Eg:

```
[4, 16, 36, 64, 100]
```

```
words=["Balaiah","Nag","Venkatesh","Chiranjeevi"]

l=[w[0] for w in words]

print(l)


Output['B', 'N', 'V', 'C']
```

Q. Write a program to display unique vowels present in the given word?

```
vowels=['a','e','i','o','u']
word=input("Enter the word to search for vowels: ")
found=[]
for letter in word:
    if letter in vowels:
        if letter not in found:
            found.append(letter)
print(found)
print("The number of different vowels present in",word,"is",len(found))
```

Tuple Data Structure

1. Tuple is exactly same as List except that it is immutable. i.e once we creates Tuple object,we cannot perform any changes in that object.

Hence Tuple is Read Only version of List.

2.If our data is fixed and never changes then we should go for Tuple.

3.Insertion Order is preserved

4. Duplicates are allowed
5. Heterogeneous objects are allowed.
6. We can preserve insertion order and we can differentiate duplicate objects by using index. Hence index will play very important role in Tuple also.

Tuple support both +ve and -ve index. +ve index means forward direction(from left to right) and -ve index means backward direction(from right to left)

7.We can represent Tuple elements within Parenthesis and with comma seperator. Parenethesis are optional but recommended to use.

Eg:

```
1.  t=10,20,30,40
2. print(t)
3. print(type(t))
4.
5. Output
6. (10, 20, 30, 40)
7. <class 'tuple'>
8.
9.    t=()
10. print(type(t)) # tuple
```

Note: We have to take special care about single valued tuple.compulsary the value should ends with comma,otherwise it is not treated as tuple.

Eg:

```
1.  t=(10)
2. print(t)
3. print(type(t))
4.

6. 10
```

Eg:

```
t=(10,)
print(t)

print(type(t))
```

Output

```
(10,)
```

Q. Which of the following are valid tuples?

1. t=()
2. t=10,20,30,40
3. t=10
4. t=10,
5. t=(10)
6. t=(10,)
7. t=(10,20,30,40)

Tuple creation:

1. t=()

creation of empty tuple

2. t=(10,)

t=10,

creation of single valued tuple ,parenthesis are optional,should ends with comma

3. t=10,20,30

t=(10,20,30)

creation of multi values tuples & parenthesis are optional

4. By using tuple() function:

```
list=[10,20,30]
        t=tuple(list)
        print(t)

        t=tuple(range(10,20,2))
        print(t)
```

Accessing elements of tuple:

We can access either by index or by slice operator

1. By using index:

```
t=(10,20,30,40,50,60)
print(t[0]) #10
print(t[-1]) #60
print(t[100])      IndexError: tuple index out of range
```

2. By using slice operator:

```
t=(10,20,30,40,50,60)
print(t[2:5])
print(t[2:100])
print(t[::2])

Output
(30, 40, 50)
(30, 40, 50, 60)
(10, 30, 50)
```

Tuple vs immutability:

Once we creates tuple,we cannot change its content.
Hence tuple objects are immutable.

Eg:

t=(10,20,30,40)

t[1]=70 TypeError: 'tuple' object does not support item assignment

Mathematical operators for tuple:

We can apply + and * operators for tuple

1. Concatenation Operator(+):

```
1.t1=(10,20,30)
t2=(40,50,60)
        t3=t1+t2
print(t3) # (10,20,30,40,50,60)
```

2. Multiplication operator or repetition operator(*)

```
t1=(10,20,30)
t2=t1*3
print(t2) #(10,20,30,10,20,30,10,20,30)
```

Important functions of Tuple:

1. len()

To return number of elements present in the tuple

Eg:

```
t=(10,20,30,40)
print(len(t)) #4
```

2. count()

To return number of occurrences of given element in the tuple

Eg:

```
t=(10,20,10,10,20)
print(t.count(10)) #3
```

3. index()

returns index of first occurrence of the given element.

If the specified element is not available then we will get ValueError.

Eg:

```
t=(10,20,10,10,20)
print(t.index(10)) #0
print(t.index(30)) ValueError: tuple.index(x): x not in tuple
```

4. sorted()

To sort elements based on default natural sorting order

```
t=(40,10,30,20)
t1=sorted(t)
print(t1)
print(t)

  Output
  [10, 20, 30, 40]
```

8. (40, 10, 30, 20)

We can sort according to reverse of default natural sorting order as follows

t1=sorted(t,reverse=True)
print(t1) [40, 30, 20, 10]

5. min() and max() functions:

These functions return min and max values according to default natural sorting

order. Eg:

```
1.  t=(40,10,30,20)
2. print(min(t)) #10
3. print(max(t)) #40
```

6. cmp():

It compares the elements of both
tuples. If both tuples are equal then
returns 0

If the first tuple is less than second tuple then it returns -1

If the first tuple is greater than second tuple then it returns

+1 Eg:

```
1.  t1=(10,20,30)
2. t2=(40,50,60)
3. t3=(10,20,30)
4. print(cmp(t1,t2)) # -1
5. print(cmp(t1,t3)) # 0
6. print(cmp(t2,t3)) # +1
```

Note: cmp() function is available only in Python2 but not in Python 3

Tuple Packing and Unpacking:

We can create a tuple by packing a group of

variables. Eg:

a=1
0
b=2
0
c=30

d=4
0

t=a,b,c,d

print(t) #(10, 20, 30, 40)

Here a,b,c,d are packed into a tuple t. This is nothing but tuple packing.

Tuple unpacking is the reverse process of tuple packing

We can unpack a tuple and assign its values to different

variables Eg:

```
t=(10,20,30,40)
a,b,c,d=t
print("a=",a,"b=",b,"c=",c,"d=",d)

Output
a= 10 b= 20 c= 30 d= 40
```

Note: At the time of tuple unpacking the number of variables and number of values should be same. ,otherwise we will get ValueError.

Eg:

```
t=(10,20,30,40)
a,b,c=t          #ValueError: too many values to unpack (expected 3)
```

Tuple Comprehension:

Tuple Comprehension is not supported by

Python. t= (x**2 for x in range(1,6))

Here we are not getting tuple object and we are getting generator object.

```
t= ( x**2 for x in range(1,6))

for x in t:

    print(x)

Output

D:\Python_classes>py test.py
1
4
9
16
25
```

Differences between List and Tuple:

List and Tuple are exactly same except small difference: List objects are mutable where as Tuple objects are immutable.

In both cases insertion order is preserved, duplicate objects are allowed, heterogenous objects are allowed, index and slicing are supported.

List	Tuple
1) List is a Group of Comma separeated Values within Square Brackets and Square Brackets are mandatory. Eg: i = [10, 20, 30, 40]	1) Tuple is a Group of Comma separeated Values within Parenthesis and Parenthesis are optional. Eg: t = (10, 20, 30, 40) t = 10, 20, 30, 40
2) List Objects are Mutable i.e. once we creates List Object we can perform any changes in that Object. Eg: i[1] = 70	2) Tuple Objeccts are Immutable i.e. once we creates Tuple Object we cannot change its content. t[1] = 70 ValueError: tuple object does not support item assignment.
3) If the Content is not fixed and keep on changing then we should go for List.	3) If the content is fixed and never changes then we should go for Tuple.
4) List Objects can not used as Keys for Dictionries because Keys should be Hashable and Immutable.	4) Tuple Objects can be used as Keys for Dictionries because Keys should be Hashable and Immutable.


python™

Set Data Structure

- If we want to represent a group of unique values as a single entity then we should go for set.
- Duplicates are not allowed.
- Insertion order is not preserved.But we can sort the elements.
- Indexing and slicing not allowed for the set.
- Heterogeneous elements are allowed.
- Set objects are mutable i.e once we creates set object we can perform any changes in that object based on our requirement.
- We can represent set elements within curly braces and with comma seperation
- We can apply mathematical operations like union,intersection,difference etc on set objects.

Creation of Set objects:

Eg:

```
1.  s={10,20,30,40}
2. print(s)
3. print(type(s))
4.

6. {40, 10, 20, 30}
```

We can create set objects by using set()

function s=set(any sequence)

Eg 1:

```
1.  l = [10,20,30,40,10,20,10]
2. s=set(l)
3. print(s) # {40, 10, 20, 30}
```

Eg 2:

```
1.  s=set(range(5))
2. print(s) #{0, 1, 2, 3, 4}
```

Note: While creating empty set we have to take special care.

Compulsory we should use set() function.

s={} ==>It is treated as dictionary but not empty set.

Eg:

```
1  s={}
2  print(s)
3  print(type(s))
4
5  Output
6  {}
7  <class 'dict'>
```

Eg:

```
1  s=set()
2  print(s)
3  print(type(s))
4
5  Output
6  set()
7  <class 'set'>
```

Important functions of set:

1. add(x):

Adds item x to the set

Eg:

```
1.  s={10,20,30}
2. s.add(40);
3. print(s) #{40, 10, 20, 30}
```

2. update(x,y,z):

To add multiple items to the set.

Arguments are not individual elements and these are Iterable objects like List,range etc. All elements present in the given Iterable objects will be added to the set.

Eg:

```
1.  s={10,20,30}
2. l=[40,50,60,10]
3. s.update(l,range(5))
4. print(s)
```

```
5.
6. Output
7.              {0, 1, 2, 3, 4, 40, 10, 50, 20, 60, 30}
```

Q. What is the difference between add() and update() functions in set?

We can use add() to add individual item to the Set,where as we can use update() function to add multiple items to Set.

add() function can take only one argument where as update() function can take any number of arguments but all arguments should be iterable objects.

Q. Which of the following are valid for set s?

1.s.add(10)
2.s.add(10,20,30) TypeError: add() takes exactly one argument (3 given)
3.s.update(10) TypeError: 'int' object is not iterable
4.s.update(range(1,10,2),range(0,10,2))

3. copy():

Returns copy of the
set. It is cloned object.

s={10,20,30}

s1=s.copy(
) print(s1)

4. pop():

It removes and returns some random element from the

set. Eg:

```
1.  s={40,10,30,20}
2. print(s)
3.  print(s.pop())
4. print(s)
5.
6. Output
7. {40, 10, 20, 30}
8. 40
9. {10, 20, 30}
```

5. remove(x):

It removes specified element from the set.

If the specified element not present in the Set then we will get KeyError

s={40,10,30,20}

s.remove(30)

print(s) # {40, 10, 20}

s.remove(50) ==>KeyError: 50

6. clear():

To remove all elements from the Set.

```
s={10,20,30}
print(s)
s.clear()
print(s)

Output
{10, 20, 30}
set()
```

Mathematical operations on the Set:

1. union():

x.union(y) ==>We can use this function to return all elements present in both sets x.union(y) or x|y

Eg:

```
x={10,20,30,40}
y={30,40,50,60}
print(x.union(y))          #{10, 20, 30, 40, 50, 60}
print(x|y)                 #{10, 20, 30, 40, 50, 60}
```

2. intersection():

x.intersection(y) or x&y

Returns common elements present in both x and y Eg:

```
x={10,20,30,40}
y={30,40,50,60}
print(x.intersection(y))          #{40, 30}
print(x&y)                 #{40, 30}
```

3. difference():

x.difference(y) or x-y

returns the elements present in x but not in y Eg:

```
x={10,20,30,40}
y={30,40,50,60}
print(x.difference(y)) #{10, 20}
print(x-y)                 #{10, 20}
```

```
print(y-x)              #{50, 60}
```

4. symmetric_difference():

x. symmetric_difference(y) or x^y

Returns elements present in either x or y but not in both Eg:

```
x={10,20,30,40}
y={30,40,50,60}
print(x.symmetric_difference(y))          #{10, 50, 20, 60}
print(x^y)          #{10, 50, 20, 60}
```

Membership operators: (in , not in)

Eg:

```
1.  s=set("durga")
2. print(s)
3. print('d' in s)
4. print('z' in s)
5.
6. Output
7. {'u', 'g', 'r', 'd', 'a'}
8. True
9. False
```

Set Comprehension:

Set comprehension is possible.

```
s={x*x for x in
range(5)} print(s) #{0, 1,
4, 9, 16}


 s={2**x for x in
range(2,10,2)} print(s)    #{16,
256, 64, 4}
```

set objects won't support indexing and slicing:

Eg:

s={10,20,30,40}

print(s[0]) ==>TypeError: 'set' object does not support indexing print(s[1:3]) ==>TypeError: 'set' object is not subscriptable

Q.Write a program to eliminate duplicates present in the list?

Approach-1:

```
l=eval(input("Enter List of values: "))
s=set(l)
print(s)

Output
D:\Python_classes>py test.py
Enter List of values: [10,20,30,10,20,40]
{40, 10, 20, 30}
```

Approach-2:

```
l=eval(input("Enter List of values: "))
l1=[]
for x in l:
    if x not in l1:
                l1.append(x)
print(l1)

Output
D:\Python_classes>py test.py
Enter List of values: [10,20,30,10,20,40]
[10, 20, 30, 40]
```

Q. Write a program to print different vowels present in the given word?

```
w=input("Enter word to search for vowels: ")
s=set(w)
v={'a','e','i','o','u'}
d=s.intersection(v)
print("The different vowel present in",w,"are",d)

Output
D:\Python_classes>py test.py
Enter word to search for vowels: durga
The different vowel present in durga are {'u', 'a'}
```

Dictionary Data Structure

We can use List,Tuple and Set to represent a group of individual objects as a single entity.

If we want to represent a group of objects as key-value pairs then we should go for Dictionary.

Eg:

rollno name

phone number--address
ipaddress
domain name

Duplicate keys are not allowed but values can be duplicated.
Hetrogeneous objects are allowed for both key and values.
insertion order is not preserved

Dictionaries are
mutable Dictionaries
are dynamic

indexing and slicing concepts are not applicable

Note: In C++ and Java Dictionaries are known as "Map" where as in Perl and Ruby it is known as "Hash"

How to create Dictionary?

d={} or d=dict()

we are creating empty dictionary. We can add entries as follows

```
d[100]="durga"
d[200]="ravi"
d[300]="shiva"
```

print(d) #{100: 'durga', 200: 'ravi', 300: 'shiva'}

If we know data in advance then we can create dictionary as follows

d={100:'durga' ,200:'ravi', 300:'shiva'}

d={key:value, key:value}

How to access data from the dictionary?

We can access data by using keys.

```
d={100:'durga' ,200:'ravi',
300:'shiva'} print(d[100])      #durga
print(d[300])           #shiva
```

If the specified key is not available then we will get

KeyError print(d[400]) # KeyError: 400

We can prevent this by checking whether key is already available or not by using has_key() function or by using in operator.

d.has_key(400) ==> returns 1 if key is available otherwise returns 0

But has_key() function is available only in Python 2 but not in Python 3. Hence compulsory we have to use in operator.

```
if 400 in d:
        print(d[400])
```

Q. Write a program to enter name and percentage marks in a dictionary and display information on the screen

```
1)  rec={}
2)  n=int(input("Enter number of students: "))
3)  i=1
4)  while i <=n:
5)      name=input("Enter Student Name: ")
6)      marks=input("Enter % of Marks of Student: ")
7)      rec[name]=marks
8)      i=i+1
9)  print("Name of Student","\t","% of marks")
10) for x in rec:
11)     print("\t",x,"\t\t",rec[x])
12)
13) Output
14) D:\Python_classes>py test.py
15) Enter number of students: 3
16) Enter Student Name: durga
17) Enter % of Marks of Student: 60%
18) Enter Student Name: ravi
19) Enter % of Marks of Student: 70%
```

20) Enter Student Name: shiva

21) Enter % of Marks of Student: 80%
22) Name of Student % of marks
23) 60%
durga
24) ravi 70 %
25) shiva 80%

How to update dictionaries?

d[key]=value

If the key is not available then a new entry will be added to the dictionary with the specified key-value pair

If the key is already available then old value will be replaced with new

value. Eg:

```
1. d={100:"durga",200:"ravi",300:"shiva"}
2. print(d)
3. d[400]="pavan"
4. print(d)
5. d[100]="sunny"
6. print(d)
7.
8. Output
9. {100: 'durga', 200: 'ravi', 300: 'shiva'}
10. {100: 'durga', 200: 'ravi', 300: 'shiva', 400: 'pavan'}
11. {100: 'sunny', 200: 'ravi', 300: 'shiva', 400: 'pavan'}
```

How to delete elements from dictionary?

del d[key]

It deletes entry associated with the specified key. If the key is not available then we will get KeyError

Eg:

```
1. d={100:"durga",200:"ravi",300:"shiva"}
2. print(d)
3. del d[100]
4. print(d)
5. del d[400]
6.
```

```
Output
{100: 'durga', 200: 'ravi', 300: 'shiva'}
```

```
{200: 'ravi', 300: 'shiva'}
KeyError: 400
```

d.clear()

To remove all entries from the dictionary Eg:

```
1.d={100:"durga",200:"ravi",300:"shiva"}
2. print(d)
3. d.clear()
4. print(d)
5.
6. Output
7. {100: 'durga', 200: 'ravi', 300: 'shiva'}
8.  {}
```

del d

To delete total dictionary.Now we cannot access d Eg:

```
1.  d={100:"durga",200:"ravi",300:"shiva"}
2. print(d)
3. del d
4. print(d)
5.
6. Output
7. {100: 'durga', 200: 'ravi', 300: 'shiva'}
8. NameError: name 'd' is not defined
```

Important functions of dictionary:

1. dict():

To create a dictionary

d=dict() ===>It creates empty dictionary

d=dict({100:"durga",200:"ravi"}) ==>It creates dictionary with specified elements
d=dict([(100,"durga"),(200,"shiva"),(300,"ravi")])==>It creates dictionary with the given list of tuple elements

2. len()

Returns the number of items in the dictionary

3. clear():

To remove all elements from the dictionary

4. get():

To get the value associated with the

key d.get(key)

If the key is available then returns the corresponding value otherwise returns None.It wont raise any error.

d.get(key,defaultvalue)

If the key is available then returns the corresponding value otherwise returns default value.

Eg:

```
d={100:"durga",200:"ravi",300:"shiva"}
print(d[100]) ==>durga
print(d[400]) ==>KeyError:400
print(d.get(100)) ==durga
print(d.get(400)) ==>None
print(d.get(100,"Guest")) ==durga
print(d.get(400,"Guest"))
==>Guest
```

3. pop():

d.pop(key)

It removes the entry associated with the specified key and returns the corresponding value

If the specified key is not available then we will get

KeyError Eg:

```
1) d={100:"durga",200:"ravi",300:"shiva"}
2) print(d.pop(100))
```

```
3) print(d)
4) print(d.pop(400))
5)
6) Output
```

```
7) durga
8) {200: 'ravi', 300: 'shiva'}
9) KeyError: 400
```

4. popitem():

It removes an arbitrary item(key-value) from the dictionaty and returns it. Eg:

```
1)  d={100:"durga",200:"ravi",300:"shiva"}
2) print(d)
3) print(d.popitem())
4) print(d)
5)
6) Output
7) {100: 'durga', 200: 'ravi', 300: 'shiva'}
8) (300, 'shiva')
9) {100: 'durga', 200: 'ravi'}
```

If the dictionary is empty then we will get
KeyError d={}

print(d.popitem()) ==>KeyError: 'popitem(): dictionary is empty'

5. keys():

It returns all keys associated eith

dictionary Eg:

```
1)  d={100:"durga",200:"ravi",300:"shiva"}
2) print(d.keys())
3) for k in d.keys():
4)       print(k)
5)
6) Output
7) dict_keys([100, 200, 300])
8) 100
9) 200
10) 300
```

6. values():

It returns all values associated with the dictionary

Eg:

```
d={100:"durga",200:"ravi",300:"shiva"}
print(d.values())
for v in d.values():
    print(v)

Output
dict_values(['durga', 'ravi', 'shiva'])
durga
ravi
shiva
```

7. items():

It returns list of tuples representing key-value pairs. [(k,v),

(k,v),(k,v)]

Eg:

```
d={100:"durga",200:"ravi",300:"shiva"}
for k,v in d.items():
    print(k,"--",v)

Output
100 -- durga
200 -- ravi
300 -- shiva
```

8. copy():

To create exactly duplicate dictionary(cloned copy)

d1=d.copy();

9. setdefault():

d.setdefault(k,v)

If the key is already available then this function returns the corresponding value.

If the key is not available then the specified key-value will be added as new item to the dictionary.

Eg:

```
d={100:"durga",200:"ravi",300:"shiva"}
print(d.setdefault(400,"pavan"))

print(d.setdefault(100,"sachin"))


print(d)
```

```
pavan
Output
durga
```

10. update():

d.update(x)

All items present in the dictionary x will be added to dictionary d

Q. Write a program to take dictionary from the keyboard and print the sum of values?

```
d=eval(input("Enter dictionary:"))
s=sum(d.values())
print("Sum= ",s)

Output
D:\Python_classes>py test.py
Enter dictionary:{'A':100,'B':200,'C':300}
Sum= 600
```

Q. Write a program to find number of occurrences of each letter present in the given string?

```
word=input("Enter any word: ")
d={}
for x in word:
    d[x]=d.get(x,0)+1
for k,v in d.items():
    print(k,"occurred ",v," times")

Output
D:\Python_classes>py test.py
Enter any word: mississippi
m occurred 1 times
i occurred 4 times
s occurred 4 times
```

```
14. p occurred 2 times
```

Q. Write a program to find number of occurrences of each vowel present in the given string?

```
1.  word=input("Enter any word: ")
2. vowels={'a','e','i','o','u'}
3. d={}
4. for x in word:

6.      if d[x]=d.get(x,0)+

8.      print(k,"occurred ",v," times")

10. Output

12. Enter any word: doganimaldoganimal

14. i occurred 2 times
13. a occurred 4 times
```

Q. Write a program to accept student name and marks from the keyboard and creates a dictionary. Also display student marks by taking student name as input?

```
1)   n=int(input("Enter the number of students: "))
2)  d={}
3)  for i in range(n):
4)      name=input("Enter Student Name: ")

6)      d[name]=marks

8)      name=input("Enter Student Name to get Marks: ")
7)  while True:
10)     if marks== -1:
11          print("Student Not Found")
12)     else:
13          print("The Marks of",name,"are",marks)
14)     option=input("Do you want to find another student marks[Yes|No]")
15)     if option=="No":
16          break

18)

20) D:\Python_classes>py test.py
19) Output
22) Enter Student Name: sunny
21) Enter the number of students: 5
```

```
Enter Student Name: banny
Enter Student Marks: 80
Enter Student Name: chinny
Enter Student Marks: 70
Enter Student Name: pinny
Enter Student Marks: 60
Enter Student Name: vinny
Enter Student Marks: 50
Enter Student Name to get Marks: sunny
The Marks of sunny are 90
Do you want to find another student marks[Yes|No]Yes
Enter Student Name to get Marks: durga
Student Not Found
Do you want to find another student marks[Yes|No]No
Thanks for using our application
```

Dictionary Comprehension:

Comprehension concept applicable for dictionaries also.

```
squares={x:x*x for x in range(1,6)}
print(squares)
doubles={x:2*x for x in range(1,6)}
print(doubles)

Output
{1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
{1: 2, 2: 4, 3: 6, 4: 8, 5: 10}
```

FUNCTIONS

If a group of statements is repeatedly required then it is not recommended to write these statements everytime seperately.We have to define these statements as a single unit and we can call that unit any number of times based on our requirement without rewriting.

This unit is nothing but function.

The main advantage of functions is code Reusability.

Note: In other languages functions are known as methods,procedures,subroutines

etc Python supports 2 types of functions

1. Built in Functions
2. User Defined Functions

1. Built in Functions:

The functions which are coming along with Python software automatically,are called built in functions or pre defined functions

Eg:

id()

type()
input()
eval()

etc..

2. User Defined Functions:

The functions which are developed by programmer explicitly according to business requirements ,are called user defined functions.

Syntax to create user defined functions:

```
def
  function_name(parameters) :
```

```
""" doc string"""
  ----
  -----
  return value
```

Note: While creating functions we can use 2 keywords

1.def (mandatory)
2.return (optional)

Eg 1: Write a function to print Hello

test.py:

```
1)  def wish():
2)      print("Hello Good Morning")
3)  wish()
4)  wish()
5)  wish()
```

Parameters

Parameters are inputs to the function. If a function contains parameters,then at the time of calling,compulsory we should provide values otherwise,otherwise we will get error.

Eg: Write a function to take name of the student as input and print wish message by name.

```
1.   def wish(name):
2.       print("Hello",name," Good Morning")
3.  wish("Durga")
4.  wish("Ravi")

6.

8.  Hello Durga Good Morning
7.  D:\Python_classes>py test.py
```

Eg: Write a function to take number as input and print its square value

```
1.   def squareIt(number):
2.       print("The Square of",number,"is", number*number)
3.  squareIt(4)
4.  squareIt(5)
5.
6.  D:\Python_classes>py test.py
7.  The Square of 4 is 16
8.  The Square of 5 is 25
```

Return Statement:

Function can take input values as parameters and executes business logic, and returns output to the caller with return statement.

Q. Write a function to accept 2 numbers as input and return sum.

```
def add(x,y):
    return x+y
result=add(10,20)
print("The sum is",result)
print("The sum is",add(100,200))


D:\Python_classes>py test.py
The sum is 30
The sum is 300
```

If we are not writing return statement then default return value is None

Eg:

```
def f1():

    print("Hello")

f1()

print(f1())


Output

Hello

Hello

None
```

Q. Write a function to check whether the given number is even or odd?

```
def even_odd(num):
    if num%2==0:
        print(num,"is Even Number")
    else:
        print(num,"is Odd Number")
even_odd(10)
even_odd(15)

Output
D:\Python_classes>py test.py
10 is Even Number
15 is Odd Number
```

Q. Write a function to find factorial of given number?

```
1)  def fact(num):
2)      result=1
3)      while num>=1:
4)          result=result*num
5)          num=num-
6)      return result
7)  for i in range(1,5):
8)      print("The Factorial of",i,"is :",fact(i))

10) Output

12) The Factorial of 1 is : 1
test.py
14) The Factorial of 3 is : 6
```

Returning multiple values from a function:

In other languages like C,C++ and Java, function can return atmost one value. But in Python, a function can return any number of values.

Eg 1:

```
1)  def sum_sub(a,b):
2)      sum=a+b
3)      sub=a-b
4)      return sum,sub

6)  print("The Sum is :",x)

8)
7)  print("The Subtraction is :",y)
10) The Sum is : 150
```

Eg 2:

```
1)def calc(a,b):
2)      sum=a+b
3)      sub=a-b
4)      mul=a*b
5)      div=a/b
6)      return sum,sub,mul,div
7)  t=calc(100,50)
8)  print("The Results are")
```

```
9)  for i in t:
10)     print(i)
11)
12) Output
13) The Results are
14) 150
15) 50
16) 5000
17) 2.0
```

Types of arguments

```
def f1(a,b):
    -----
    -----
    -----
 f1(10,20)
```

a,b are formal arguments where as 10,20 are actual

arguments There are 4 types are actual arguments are allowed

in Python.

1. positional arguments
2. keyword arguments
3. default arguments
4. Variable length arguments

1. positional arguments:

These are the arguments passed to function in correct positional

order. def sub(a,b):

```
  print(a-b)

sub(100,200)
sub(200,100)
```

The number of arguments and position of arguments must be matched. If we change the order then result may be changed.

If we change the number of arguments then we will get error.

2. keyword arguments:

We can pass argument values by keyword i.e by parameter name. Eg:

```
1.  def wish(name,msg):
2.      print("Hello",name,msg)
3. wish(name="Durga",msg="Good Morning")
4. wish(msg="Good Morning",name="Durga")
5.
6. Output
7. Hello Durga Good Morning
8. Hello Durga Good Morning
```

Here the order of arguments is not important but number of arguments must be matched. Note:

We can use both positional and keyword arguments simultaneously. But first we have to take positional arguments and then keyword arguments,otherwise we will get syntaxerror.

```
def wish(name,msg):
        print("Hello",name,msg)
wish("Durga","GoodMorning")                ==>valid
wish("Durga",msg="GoodMorning")            ==>valid
wish(name="Durga","GoodMorning") ==>invalid
SyntaxError: positional argument follows keyword
argument
```

3. Default Arguments:

Sometimes we can provide default values for our positional arguments. Eg:

```
1)  def wish(name="Guest"):
2)      print("Hello",name,"Good Morning")
3)
4) wish("Durga")

5) wish()

8) Hello Durga Good Morning
7) Output
```

If we are not passing any name then only default value will be considered.

***Note:

After default arguments we should not take non default

arguments def wish(name="Guest",msg="Good Morning"):

===>Valid

def wish(name,msg="Good Morning"):
===>Valid def wish(name="Guest",msg):
==>Invalid

SyntaxError: non-default argument follows default argument

4. Variable length arguments:

Sometimes we can pass variable number of arguments to our function,such type of arguments are called variable length arguments.

We can declare a variable length argument with * symbol as

follows def f1(*n):

We can call this function by passing any number of arguments including zero number. Internally all these values represented in the form of tuple.

Eg:

```
1)  def sum(*n):
2)      total=0
3)      for n1 in n:
4)          total=total+n1
5)      print("The Sum=",total)
6)
7)  sum()
8)  sum(10)
9)  sum(10,20)
10) sum(10,20,30,40)
11)
12) Output
13) The Sum= 0
14) The Sum= 10
15) The Sum= 30
16) The Sum= 100
```

Note:

We can mix variable length arguments with positional arguments.

Eg:

```
def f1(n1,*s):
    print(n1)

    for s1 in s:
        print(s1)

f1(10)
f1(10,"A",30,"B")
f1(10,20,30,40)
```

Output

```
10

30
10
10

30
```

Note: After variable length argument,if we are taking any other arguments then we should provide values as keyword arguments.

Eg:

```
def f1(*s,n1):
    for s1 in s:
        print(s1)
    print(n1)

f1("A","B",n1=10)
```

Output

```
A
B
10
```

f1("A","B",10) ==>Invalid

TypeError: f1() missing 1 required keyword-only argument: 'n1'

Note: Function vs Module vs Library:

1. A group of lines with some name is called a function
2. A group of functions saved to a file , is called Module
3. A group of Modules is nothing but Library

Function

2. Local Variables:

The variables which are declared inside a function are called local variables.

Local variables are available only for the function in which we declared it.i.e from outside of function we cannot access.

Eg:

```
1)  def f1():
2)      a=10

3)      print(a)  # valid

4)      print(a)    #invalid

5)  f1()
6)
7)
```

global keyword:

We can use global keyword for the following 2 purposes:

1. To declare global variable inside function
2. To make global variable available to the function so that we can perform required modifications

Eg 1:

```
1)  a=10
2)  def f1():
3)      a=777
4)      print(a)
5)
6)  def f2():
7)      print(a)
8)
9)  f1()
10) f2()
11)
12) Output
13) 777
14) 10
```

Eg 2:

```
a=10
def f1():

    a=777


    print(a)
print(a)
def f2():
f1()


a
777
f2()
```

Eg 3:

```
def f1():
    a=10


    print(a)

print(a)
def f2():
f1()
```

Eg 4:

```
def f1():
    global a
    a=10
    print(a)

def f2():
    print(a)

f1()
f2()

Output
10
10
```

Note: If global variable and local variable having the same name then we can access global variable inside a function as follows

```
1)  a=10      #global variable
2)  def f1():
3)      a=777 #local variable
4)      print(a)

6)  f1()

8)
7)
10) 777
```

Recursive Functions

A function that calls itself is known as Recursive

Function. Eg:

factorial(3)=3*factorial(2)

=3*2*factorial(1)

=3*2*1*factorial(0)

=3*2*1*1

=6

factorial(n)= n*factorial(n-1)

The main advantages of recursive functions are:

1. We can reduce length of the code and improves readability
2. We can solve complex problems very easily.

Q. Write a Python Function to find factorial of given number with recursion.

Eg:

```
1)def factorial(n):
2)      if n==0:
3)          result=1
4)      else:
5)          result=n*factorial(n-1)
6)      return result
7)  print("Factorial of 4 is :",factorial(4))
8)  print("Factorial of 5 is :",factorial(5))
9)
10) Output
```

```
11) Factorial of 4 is : 24
12) Factorial of 5 is : 120
```

Anonymous Functions:

Sometimes we can declare a function without any name,such type of nameless functions are called anonymous functions or lambda functions.

The main purpose of anonymous function is just for instant use(i.e for one time usage)

Normal Function:

We can define by using def
keyword. def squarelt(n):

    return n*n

lambda Function:

We can define by using lambda

keyword lambda n:n*n

Syntax of lambda Function:

lambda argument_list : expression

Note: By using Lambda Functions we can write very concise code so that readability of the program will be improved.

Q. Write a program to create a lambda function to find square of given number?

```
1)  s=lambda n:n*n
2)  print("The Square of 4 is :",s(4))

4)  print("The Square of 5 is :",s(5))

6)  The Square of 4 is : 16
5)  Output
```

Q. Lam/bda function to find sum of 2 given numbers

```
1)  s=lambda a,b:a+b
2)  print("The Sum of 10,20 is:",s(10,20))
```

```
3) print("The Sum of 100,200 is:",s(100,200))
4)

6) The Sum of 10,20 is: 30
```

Function Aliasing:

For the existing function we can give another name, which is nothing but function aliasing.

Eg:

```
1)  def wish(name):
2)      print("Good Morning:",name)
3)
4) greeting=wish
5) print(id(wish))
6) print(id(greeting))
7)
8) greeting('Durga')
9) wish('Durga')
```

Output

```
4429336
4429336
Good Morning: Durga
Good Morning: Durga
```

Note: In the above example only one function is available but we can call that function by using either wish name or greeting name.

If we delete one name still we can access that function by using alias name

Eg:

```
1)  def wish(name):
2)      print("Good Morning:",name)
```

```
3)
4) greeting=wish

6) greeting('Durga')

8)
7) wish('Durga')
10) #wish('Durga') ==>NameError: name 'wish' is not defined
```

Output

```
Good Morning:
Durga Good
Morning: Durga
Good Morning: Pavan
```

Nested Functions:

We can declare a function inside another function, such type of functions are called Nested functions.

Eg:

```
1)  def outer():
2)      print("outer function started")
3)      def inner():
4)          print("inner function execution")
5)      print("outer function calling inner function")
6)      inner()
7)  outer()
8)  #inner()       ==>NameError: name 'inner' is not defined
```

Output

```
outer function started
outer function calling inner function
inner function execution
```

In the above example inner() function is local to outer() function and hence it is not possible to call directly from outside of outer() function.

Note: A function can return another function.

Eg:

```
1)  def outer():
2)      print("outer function started")
3)      def inner():
4)          print("inner function execution")
```

```
5)                  print("outer function returning inner function")
6)      return inner
7)  f1=outer()
8)  f1()
9)  f1()
10) f1()
```

Output

```
outer function started
outer function returning inner function
inner function execution
inner function execution
inner function execution
```

Q. What is the differenece between the following lines?

```
f1 = outer
```

```
f1 = outer()
```

- In the first case for the outer() function we are providing another name f1(function aliasing).
- But in the second case we calling outer() function,which returns inner function.For that inner function() we are providing another name f1

Modules

A group of functions, variables and classes saved to a file, which is nothing but module. Every Python file (.py) acts as a module.

Eg: durgamath.py

```
1)  x=888
2)
4)    print("The Sum:",a+b)
6)  def product(a,b):
5)
```

durgamath module contains one variable and 2 functions.

If we want to use members of module in our program then we should import that module. import modulename

We can access members by using module name. modulename.variable

modulename.function()

test.py:

```
1)  import durgamath
2)  print(durgamath.x)
3)  durgamath.add(10,20)
4)  durgamath.product(10,20)
5)
6)    Output
7)                  888
8)  The Sum: 30
9)  The Product: 200
```

Note:

whenever we are using a module in our program, for that module compiled file will be generated and stored in the hard disk permanently.

Renaming a module at the time of import (module aliasing):

Eg:

import durgamath as m

here durgamath is original module name and m is alias name.

We can access members by using alias name

m test.py:

```
1)  import durgamath as m
2)  print(m.x)
3)  m.add(10,20)
4)  m.product(10,20)
```

from ... import:

We can import particular members of module by using from ... import .

The main advantage of this is we can access members directly without using module name.

Eg:

```
from durgamath import x,add
print(x)

add(10,20)

product(10,20)==> NameError: name 'product' is not defined
```

We can import all members of a module as follows from durgamath import *

test.py:

```
1)  from durgamath import *
2)  print(x)
3)  add(10,20)
4)  product(10,20)
```

Various possibilties of import:

import modulename

import
module1,module2,module3
import module1 as m

import module1 as m1,module2 as
m2,module3 from module import member

from module import
member1,member2,memebr3 from module
import memeber1 as x

from module import *

member aliasing:

from durgamath import x as y,add as
sum print(y)

sum(10,20)

Once we defined as alias name,we should use alias name only and we should not use original name

Eg:

from durgamath import x as y
print(x)==>NameError: name 'x' is not defined

Reloading a Module:

By default module will be loaded only once eventhough we are importing multiple multiple times.

Demo Program for module reloading:

```
1)  import time
2)  from imp import reload
3)  import module1
4)  time.sleep(30)
5)  reload(module1)
6)  time.sleep(30)
7)  reload(module1)
8)  print("This is test file")
```

Note: In the above program, everytime updated version of module1 will be available to our program

module1.py:

```
print("This is from module1")
```

test.py

```
import module1
import module1
import module1
import module1

print("This is test module")
```

```
This is from module1
Output
```

In the above program test module will be loaded only once eventhough we are importing multiple times.

The problem in this approach is after loading a module if it is updated outside then updated version of module1 is not available to our program.

We can solve this problem by reloading module explicitly based on our requirement. We can reload by using reload() function of imp module.

```
import imp
imp.reload(module1)
```

test.py:

```
import module1
import module1
from imp import reload
reload(module1)
reload(module1)
reload(module1)
print("This is test module")
```

In the above program module1 will be loaded 4 times in that 1 time by default and 3 times explicitly. In this case output is

```
This is from module1
This is from module1
This is test module
```

The main advantage of explicit module reloading is we can ensure that updated version is always available to our program.

Finding members of module by using dir() function:

Python provides inbuilt function dir() to list out all members of current module or a specified module.

dir() ===>To list out all members of current module
dir(moduleName)==>To list out all members of specified module

Eg 1: test.py

```
1)  x=10
2)  y=20
3)  def f1():
4)      print("Hello")
5)  print(dir())  # To print all members of current module
6)
7)  Output
8)  ['__annotations__', '__builtins__', '__cached__', '__doc__', '__file__', '__loader__', '__nam
     e__', '__package__', '__spec__', 'f1', 'x', 'y']
```

Eg 2: To display members of particular

module: durgamath.py:

```
1)  x=888
2)
4)    print("The Sum:",a+b)
6)  def product(a,b):
```

test.py:

```
1)  import durgamath
2)  print(dir(durgamath))
3)
4)  Output
5)  ['__builtins__', '__cached__', '__doc__', '__file__', '__loader__', '__name__',
6)  '__package__', '__spec__', 'add', 'product', 'x']
```

Note: For every module at the time of execution Python interpreter will add some special properties automatically for internal use.

Eg: builtins , _cached , doc , file , loader , name , package , _spec

Working with math module:

Python provides inbuilt module math.

This module defines several functions which can be used for mathematical operations. The main important functions are

1. sqrt(x)
2. ceil(x)
3. floor(x)
4. fabs(x)
5. log(x)
6. sin(x)
7. tan(x)

....

Eg:

```
1)  from math import *
2)  print(sqrt(4))
3)  print(ceil(10.1))
4)  print(floor(10.1))
5)  print(fabs(-10.6))
6)  print(fabs(10.6))
```

```
7)
8)  Output
9)  2.0
10) 11
11) 10
12) 10.6
13) 10.6
```

Note:

We can find help for any module by using help()

function Eg:

```
import
math
help(math)
```

Working with random module:

This module defines several functions to generate random numbers.

We can use these functions while developing games,in cryptography and to generate random numbers on fly for authentication.

1. random() function:

This function always generate some float value between 0 and 1 (not inclusive)

$$0<x<1$$

Eg:

```
1)    from random import
2)  for i in range(10):
3)        print(random()
4)
5)    Output
6)    0.4572685609302056

8)  0.15444034016553587

10) 0.1330257265904884

12) 0.6586741197891783

14) 0.25540891083913053
0.9291139798071045
```

2. randint() function:

To generate random integer beween two given

numbers(inclusive) Eg:

```
1)    from random import
2)  for i in range(10):
3)        print(randint(1,100)) # generate random int value between 1 and
4)
5)  Output
6)  51

8)  39

10) 49

9)  70
12) 52

14) 40
```

3. uniform():

It returns random float values between 2 given numbers(not

inclusive) Eg:

```
1)    from random import
2)  for i in range(10):
3)        print(uniform(1,10)
4)
5)  Output
6)  9.787695398230332

8)  8.068672144377329

10) 6.363511674803802

12) 4.822867939432386

11)
14) 7.508457735544763
```

random() ===>in between 0 and 1 (not inclusive)
randint(x,y) ==>in between x and y
(inclusive) uniform(x,y) ==> in between x and y
(not inclusive)

4. randrange([start],stop,[step])

returns a random number from range start<= x < stop

start argument is optional and default value is 0 step argument is optional and default value is 1

randrange(10)–>generates a number from 0 to 9
randrange(1,11)–>generates a number from 1 to 10
randrange(1,11,2)–>generates a number from 1,3,5,7,9

Eg 1:

```
1)  from random import
2) for i in range(10):
3)      print(randrange(10)
4)
5) Output
6) 9

8) 0

10) 9

9) 2
12) 8

14) 5
11) 4
```

Eg 2:

```
1)  from random import
2) for i in range(10):
3)      print(randrange(1,11)
4)
5) Output
6) 2

8) 8

10) 3

9) 10
12) 9

11) 5
14) 6
```

Eg 3:

```
1)   from random import
2)  for i in range(10):
3)       print(randrange(1,11,2)
4)
5)  Output
6)  1
7)  3
8)  9

10) 7

12) 1
11) 1
14) 7
```

5. choice() function:

It wont return random number.

It will return a random object from the given list or

tuple. Eg:

```
1)   from random import *
2)  list=["Sunny","Bunny","Chinny","Vinny","pinny"]
3)  for i in range(10):
4)       print(choice(list)
```

Output

Bunny
pinny
Bunny
Sunny
Bunny
pinny
pinny
Vinny
Bunny
Sunny

Packages

It is an encapsulation mechanism to group related modules into a single unit.

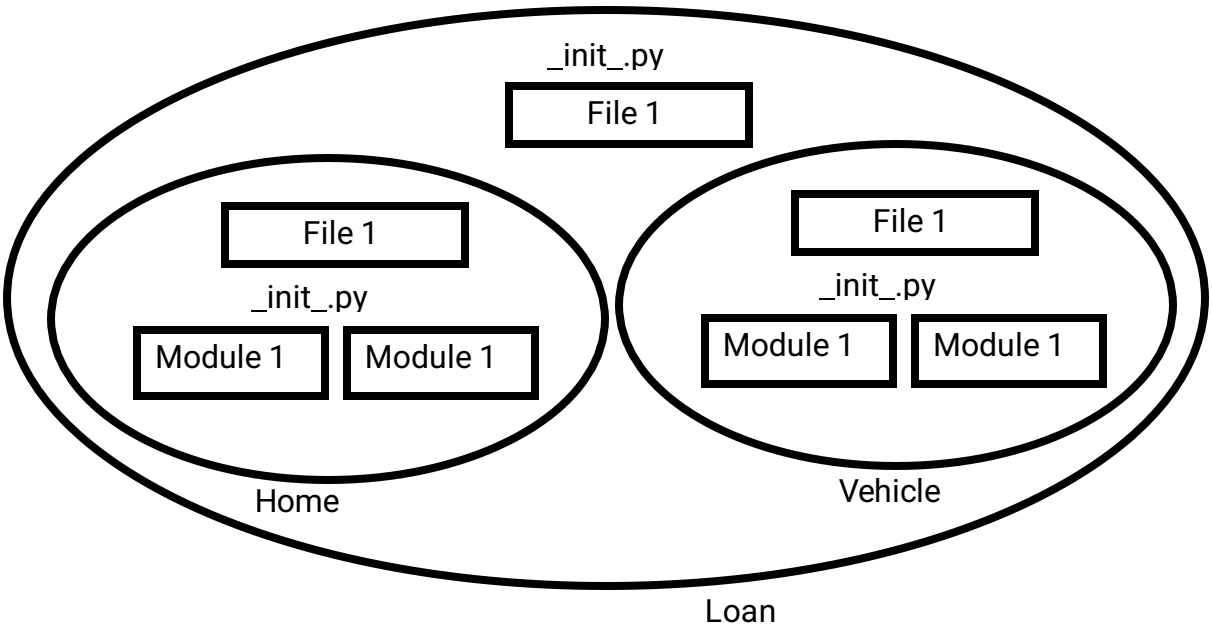
package is nothing but folder or directory which represents collection of Python modules.

Any folder or directory contains _init_.py file,is considered as a Python package.This file can be empty.

A package can contains sub packages also.

x.py y.py
y _ y


Loan



The main advantages of package statement are

1. We can resolve naming conflicts
2. We can identify our components uniquely
3. It improves modularity of the

application Eg 1:

D:\Python_classes>

|-test.py

|-pack1

|-module1.py

|-_init_.py

init.py:

empty file

module1.py:

```
def f1():
        print("Hello this is from module1 present in pack1")
```

test.py (version-1):

```
import
pack1.module1
pack1.module1.f1()
```

test.py (version-2):

```
from pack1.module1 import
f1 f1()
```

Eg 2:

```
D:\Python_classes>
    |-test.py
    |-com
        |-module1.py
            |-_init_.py
            |-durgasoft
              |-module2.py
              |-_init_.py
```

__init__.py: empty file

module1.py:

```
def f1():
        print("Hello this is from module1 present in com")
```

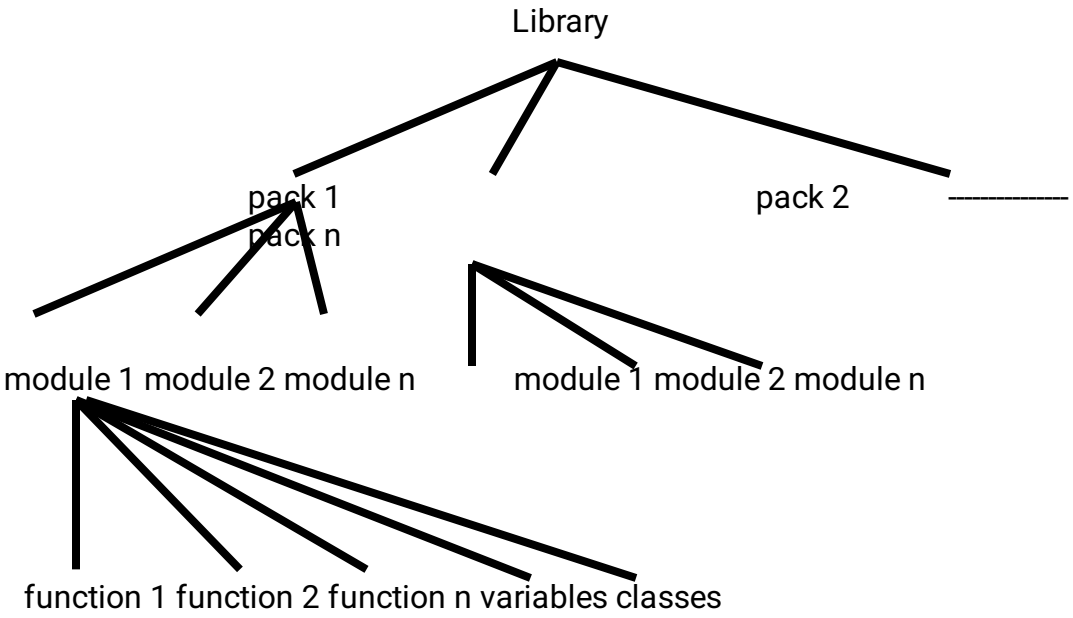
module2.py:

```
def f2():
        print("Hello this is from module2 present in com.durgasoft")
```

test.py:

```
1.  from com.module1 import f1
2. from com.durgasoft.module2 import f2
3. f1()
4. f2()
5.
6. Output
7. D:\Python_classes>py test.py
8. Hello this is from module1 present in com
9. Hello this is from module2 present in com.durgasoft
```

Note: Summary diagram of library,packages,modules which contains functions,classes and variables.


Library
pack 1
pack n
pack 2
module 1 module 2 module n
module 1 module 2 module n
function 1 function 2 function n variables classes

File Handling

As the part of programming requirement, we have to store our data permanently for future purpose. For this requirement we should go for files.

Files are very common permanent storage areas to store our data.

Types of Files:

There are 2 types of files

1. Text Files:

Usually we can use text files to store character data eg: abc.txt

2. Binary Files:

Usually we can use binary files to store binary data like images,video files, audio files etc...

Opening a File:

Before performing any operation (like read or write) on the file,first we have to open that file.For this we should use Python's inbuilt function open()

But at the time of open, we have to specify mode,which represents the purpose of opening file.

```
f = open(filename, mode)
```

The allowed modes in Python are

1. r open an existing file for read operation. The file pointer is positioned at the beginning of the file.If the specified file does not exist then we will get FileNotFoundError.This is default mode.

2. w open an existing file for write operation. If the file already contains some data then it will be overridden. If the specified file is not already avaialble then this mode will create that file.

3. a open an existing file for append operation. It won't override existing data.If the specified file is not already avaialble then this mode will create a new file.

4. r+ To read and write data into the file. The previous data in the file will not be deleted.The file pointer is placed at the beginning of the file.

5. w+ To write and read data. It will override existing data.

6. a+ To append and read data from the file.It wont override existing data.

7. x To open a file in exclusive creation mode for write operation. If the file already exists then we will get FileExistsError.

Note: All the above modes are applicable for text files. If the above modes suffixed with 'b' then these represents for binary files.

Eg: rb,wb,ab,r+b,w+b,a+b,xb

```
f = open("abc.txt","w")
```

We are opening abc.txt file for writing data.

Closing a File:

After completing our operations on the file,it is highly recommended to close the file. For this we have to use close() function.

```
f.close()
```

Various properties of File Object:

Once we opend a file and we got file object,we can get various details related to that file by using its properties.

name Name of opened file

mode Mode in which the file is opened

closed Returns boolean value indicates that file is closed or not

readable() Retruns boolean value indicates that whether file is readable or not

writable() Returns boolean value indicates that whether file is writable or not.

Eg:

```
1)  f=open("abc.txt",'w')
2)  print("File Name: ",f.name)
3)  print("File Mode: ",f.mode)
4)  print("Is File Readable: ",f.readable())
5)  print("Is File Writable: ",f.writable())
6)  print("Is File Closed : ",f.closed)
7)  f.close()
8)  print("Is File Closed : ",f.closed)
9)
10)
11) Output
12) D:\Python_classes>py test.py
13) File Name: abc.txt
14) File Mode: w
15) Is File Readable: False
16) Is File Writable: True
17) Is File Closed : False
18) Is File Closed : True
```

Writing data to text files:

We can write character data to the text files by using the following 2 methods.

write(str)

writelines(list of

lines) Eg:

```
1)  f=open("abcd.txt",'w')
2)  f.write("Micro\n")
3)  f.write("Software\n")
4)  f.write("Solutions\n")
5)  print("Data written to the file successfully")
6)    f.close()
```

abcd.txt:

Micro
Software
Solutions

Note: In the above program, data present in the file will be overridden everytime if we run the program. Instead of overriding if we want append operation then we should open the file as follows.

```
f = open("abcd.txt","a")
```

Eg 2:

```
1)  f=open("abcd.txt",'w')
2)  list=["sunny\n","bunny\n","vinny\n","chinny"]
3)  f.writelines(list)
4)  print("List of lines written to the file successfully")
5)                  f.close()
```

abcd.txt:

sunny

bunny
vinny
chinn
y

Note: while writing data by using write() methods, compulsory we have to provide line seperator(\n),otherwise total data should be written to a single line.

Reading Character Data from text files:

We can read character data from text file by using the following read methods.

read() To read total data from the file

read(n) To read 'n' characters from the file

readline() To read only one line
readlines() To read all lines into a list

Eg 1: To read total data from the file

```
1)  f=open("abc.txt",'r')
2)  data=f.read()
3)  print(data)
4)  f.close()
5)
6)  Output
7)  sunny
8)  bunny
9)  chinny
10) vinny
```

Eg 2: To read only first 10 characters:

```
1)  f=open("abc.txt",'r')
2)  data=f.read(10)
3)  print(data)
4)  f.close()
```

5)

```
6) Output
7) sunny
8) bunn
```

Eg 3: To read data line by line:

```
1) f=open("abc.txt",'r')
2) line1=f.readline()
3) print(line1,end='')
4) line2=f.readline()
5) print(line2,end='')
6) line3=f.readline()
7) print(line3,end='')
8)
9) f.close()
10)
11) Output
12) sunny
13) bunny
```

Eg 4: To read all lines into list:

```
1) f=open("abc.txt",'r')
2) lines=f.readlines()
3) for line in lines:
4)     print(line,end='')
5)
6) f.close()
7)
8) sunny
7) Output
10) chinny
```

Eg 5:

```
1) f=open("abc.txt","r")
2) print(f.read(3))
3) print(f.readline())
4) print(f.read(4))
5) print("Remaining data")
6) print(f.read())
7)
8) Output
9) sun
10) ny
11)
12) bunn
13) Remaining data
```

```
14) y
15) chinny
16) vinny
```

The with statement:

The with statement can be used while opening a file.We can use this to group file operation statements within a block.

The advantage of with statement is it will take care closing of file,after completing all operations automatically even in the case of exceptions also, and we are not required to close explicitly.

Eg:

```
1)  with open("abc.txt","w") as f:
2)      f.write("Durga\n")
3)      f.write("Software\n")
4)      f.write("Solutions\n")
5)      print("Is File Closed: ",f.closed)
6)  print("Is File Closed: ",f.closed)
7)
8)  Output
9)  Is File Closed: False
10) Is File Closed: True
```

The seek() and tell() methods:

tell():

==>We can use tell() method to return current position of the cursor(file pointer) from beginning of the file. [can you plese telll current cursor position]

The position(index) of first character in files is zero just like string

index. Eg:

```
1)  f=open("abc.txt","r")
2)  print(f.tell())
3)  print(f.read(2))
4)  print(f.tell())
5)  print(f.read(3))
6)  print(f.tell())
```

abc.txt:
sunny
bunny
chinny

vinny

Output:

0

su
 2

nny
 5

seek():

We can use seek() method to move cursor(file pointer) to specified location.
[Can you please seek the cursor to a particular location]

f.seek(offset, fromwhere)

offset represents the number of positions

The allowed values for second attribute(from where)
are 0 >From beginning of file(default value)

1 >From current position
2 >From end of the file

Note: Python 2 supports all 3 values but Python 3 supports only

zero. Eg:

```
1)  data="All Students are STUPIDS"
2)  f=open("abc.txt","w")
3)  f.write(data)
4)  with open("abc.txt","r+") as f:
5)                  text=f.read()
6)      print(text)
7)                  print("The Current Cursor Position: ",f.tell())
8)      f.seek(17)
9)                  print("The Current Cursor Position: ",f.tell())
10)     f.write("GEMS!!!")
11)                 f.seek(0)
12)     text=f.read()
13)                 print("Data After Modification:")
14)     print(text)
15)
16) Output
17)
18) All Students are STUPIDS
19) The Current Cursor Position: 24
20) The Current Cursor Position: 17
21) Data After Modification:
```

22) All Students are GEMS!!!

Handling Binary Data:

It is very common requirement to read or write binary data like images,video files,audio files etc.

Q. Program to read image file and write to a new image file?

```
1)  f1=open("rossum.jpg","rb")
2)  f2=open("newpic.jpg","wb")
3)  bytes=f1.read()
4)  f2.write(bytes)
5)  print("New Image is available with the name: newpic.jpg")
```

Handling csv files:

CSV==>Comma seperated values

As the part of programming,it is very common requirement to write and read data wrt csv files. Python provides csv module to handle csv files.

Writing data to csv file:

```
1)  import csv
2)  with open("emp.csv","w",newline='') as f:
3)      w=csv.writer(f) # returns csv writer object
4)      w.writerow(["ENO","ENAME","ESAL","EADDR"])
5)      n=int(input("Enter Number of Employees:"))
6)      for i in range(n):
7)          eno=input("Enter Employee No:")
8)          ename=input("Enter Employee Name:")
9)          esal=input("Enter Employee Salary:")
10)         eaddr=input("Enter Employee Address:")
11)         w.writerow([eno,ename,esal,eaddr])
12) print("Total Employees data written to csv file successfully")
```

Note: Observe the difference with newline attribute and

without with open("emp.csv","w",newline='') as f:

with open("emp.csv","w") as f:

Note: If we are not using newline attribute then in the csv file blank lines will be included between data. To prevent these blank lines, newline attribute is required in Python-3,but in Python-2 just we can specify mode as 'wb' and we are not required to use newline attribute.

Reading Data from csv file:

```
1)  import csv
2)  f=open("emp.csv",'r')
3)  r=csv.reader(f) #returns csv reader object
4)  data=list(r)
5)  #print(data)
6)  for line in data:
7)      for word in line:
8)          print(word,"\t",end='')
9)      print()
10)
11) Output
12) D:\Python_classes>py test.py
13) ENO ENAME ESAL  EADDR
14) 100    Durga 1000    Hyd
15) 200    Sachin 2000   Mumbai
16) 300    Dhoni 3000    Ranchi
```

Zipping and Unzipping Files:

It is very common requirement to zip and unzip
files. The main advantages are:

1. To improve memory utilization
2. We can reduce transport time
3. We can improve performance.

To perform zip and unzip operations, Python contains one in-bulit module zip
file. This module contains a class : ZipFile

To create Zip file:

We have to create ZipFile class object with name of the zip file,mode and constant ZIP_DEFLATED. This constant represents we are creating zip file.

```
f = ZipFile("files.zip","w","ZIP_DEFLATED")
```

Once we create ZipFile object,we can add files by using write() method.

```
f.write(filename)
```

Eg:

```
1)  from zipfile import *
2) f=ZipFile("files.zip",'w',ZIP_DEFLATED)
3) f.write("file1.txt")
4) f.write("file2.txt")
5) f.write("file3.txt")
6) f.close()
7) print("files.zip file created successfully")
```

To perform unzip operation:

We have to create ZipFile object as follows

```
f = ZipFile("files.zip","r",ZIP_STORED)
```

ZIP_STORED represents unzip operation. This is default value and hence we are not required to specify.

Once we created ZipFile object for unzip operation,we can get all file names present in that zip file by using namelist() method.

```
names = f.namelist()
```

Eg:

```
1)  from zipfile import *
2) f=ZipFile("files.zip",'r',ZIP_STORED)
3) names=f.namelist()
4) for name in names:
5)                 print( "File Name: ",name)
6)     print("The Content of this file is:")
7)                 f1=open(name,'r')
8)     print(f1.read())
9)                 print()
```

Pickling and Unpickling of Objects:

Sometimes we have to write total state of object to the file and we have to read total object from the file.

The process of writing state of object to the file is called pickling and the process of reading state of an object from the file is called unpickling.

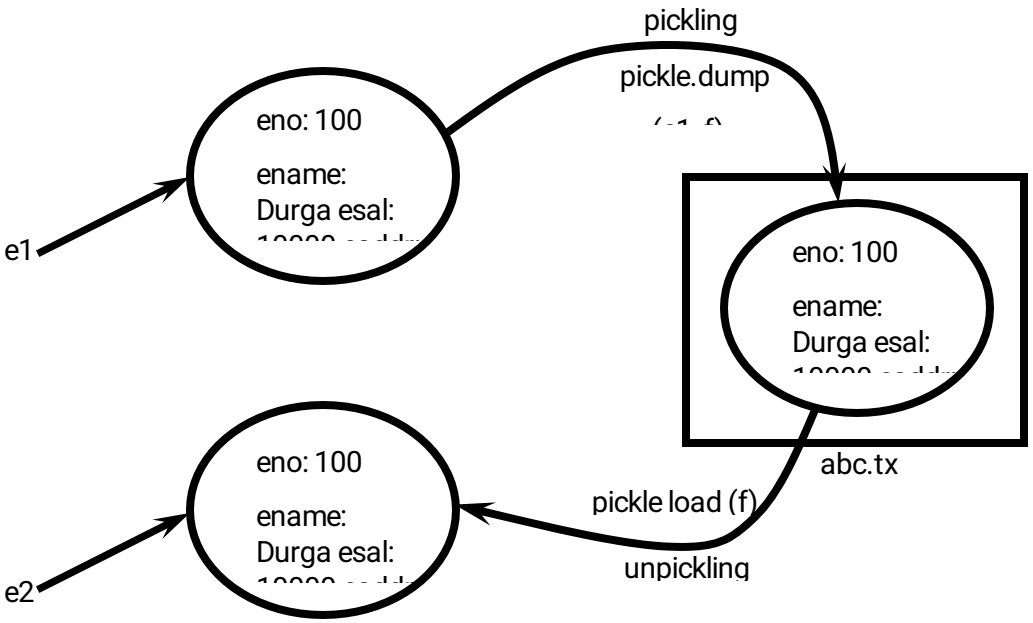
We can implement pickling and unpickling by using pickle module of

Python. pickle module contains dump() function to perform pickling.

```
pickle.dump(object,file
```

pickle module contains load() function to perform unpickling

```
obj=pickle.load(file
```



Writing and Reading State of object by using pickle Module:

```
1)  import pickle
2)  class Employee:
3)      def init (self,eno,ename,esal,eaddr):
4)          self.eno=eno;
5)          self.ename=ename;
6)          self.esal=esal;
7)          self.eaddr=eaddr;
8)      def display(self):
9)          print(self.eno,"\t",self.ename,"\t",self.esal,"\t",self.eaddr)
10) with open("emp.dat","wb") as f:
11)     e=Employee(100,"Durga",1000,"Hyd")
12)     pickle.dump(e,f)
13)     print("Pickling of Employee Object completed...")
14)
15) with open("emp.dat","rb") as f:
16)     obj=pickle.load(f)
17)     print("Printing Employee Information after unpickling")
18)     obj.display()
```

Writing Multiple Employee Objects to the file:

emp.py:

```
1)  class Employee:
2)      def init (self,eno,ename,esal,eaddr):
3)          self.eno=eno;
4)          self.ename=ename;
5)          self.esal=esal;
eaddr=eaddr;
7)   def display(self):
8)
9)      print(self.eno,"\t",self.ename,"\t",self.esal,"\t",self.eaddr)
```

pick.py:

```
1)  import emp,pickle
2)  f=open("emp.dat","wb")
3)  n=int(input("Enter The number of Employees:"))
4)  for i in range(n):
5)                  eno=int(input("Enter Employee Number:"))
6)      ename=input("Enter Employee Name:")
7)                  esal=float(input("Enter Employee Salary:"))
8)      eaddr=input("Enter Employee Address:")
9)                  e=emp.Employee(eno,ename,esal,eaddr)
10)     pickle.dump(e,f)
11) print("Employee Objects pickled successfully")
```

unpick.py:

```
1)   import emp,pickle
2)  f=open("emp.dat","rb")
3)  print("Employee Details:")
4)  while True:

6)          obj=pickle.load(f
7)          obj.display(
8)       except EOFError:
9)          print("All employees Completed")
10          break
11)
```

Exception Handling

In any programming language there are 2 types of errors are possible.

1.Syntax Errors
2.Runtime Errors

1. Syntax Errors:

The errors which occurs because of invalid syntax are called syntax errors. Eg 1:

```
x=10
if x==10
  print("Hello")
```

SyntaxError: invalid

syntax Eg 2:

```
print "Hello"
```

SyntaxError: Missing parentheses in call to 'print'

Note:

Programmer is responsible to correct these syntax errors. Once all syntax errors are corrected then only program execution will be started.

2. Runtime Errors:

Also known as exceptions.

While executing the program if something goes wrong because of end user input or programming logic or memory problems etc then we will get Runtime Errors.

Eg: print(10/0) ==>ZeroDivisionError: division by zero

print(10/"ten") ==>TypeError: unsupported operand type(s) for /: 'int' and 'str'

```
x=int(input("Enter Number:"))
```

print(x) D:

\Python_classes>py test.py

Enter Number:ten

ValueError: invalid literal for int() with base 10: 'ten'

Note: Exception Handling concept applicable for Runtime Errors but not for syntax errors

What is Exception:

An unwanted and unexpected event that disturbs normal flow of program is called exception.

Eg:

ZeroDivisionError
TypeError
ValueError
FileNotFoundError
EOFError
SleepingError
TyrePuncturedError

It is highly recommended to handle exceptions. The main objective of exception handling is Graceful Termination of the program(i.e we should not block our resources and we should not miss anything)

Exception handling does not mean repairing exception. We have to define alternative way to continue rest of the program normally.

Eg:

For example our programming requirement is reading data from remote file locating at London. At runtime if london file is not available then the program should not be terminated abnormally. We have to provide local file to continue rest of the program normally. This way of defining alternative is nothing but exception handling.

try:

```
    read data from remote file locating at london
except FileNotFoundError:
    use local file and continue rest of the program normally
```

Q. What is an Exception?

Q. What is the purpose of Exception Handling?

Q. What is the meaning of Exception Handling?

Default Exception Handing in Python:

Every exception in Python is an object. For every exception type the corresponding classes are available.

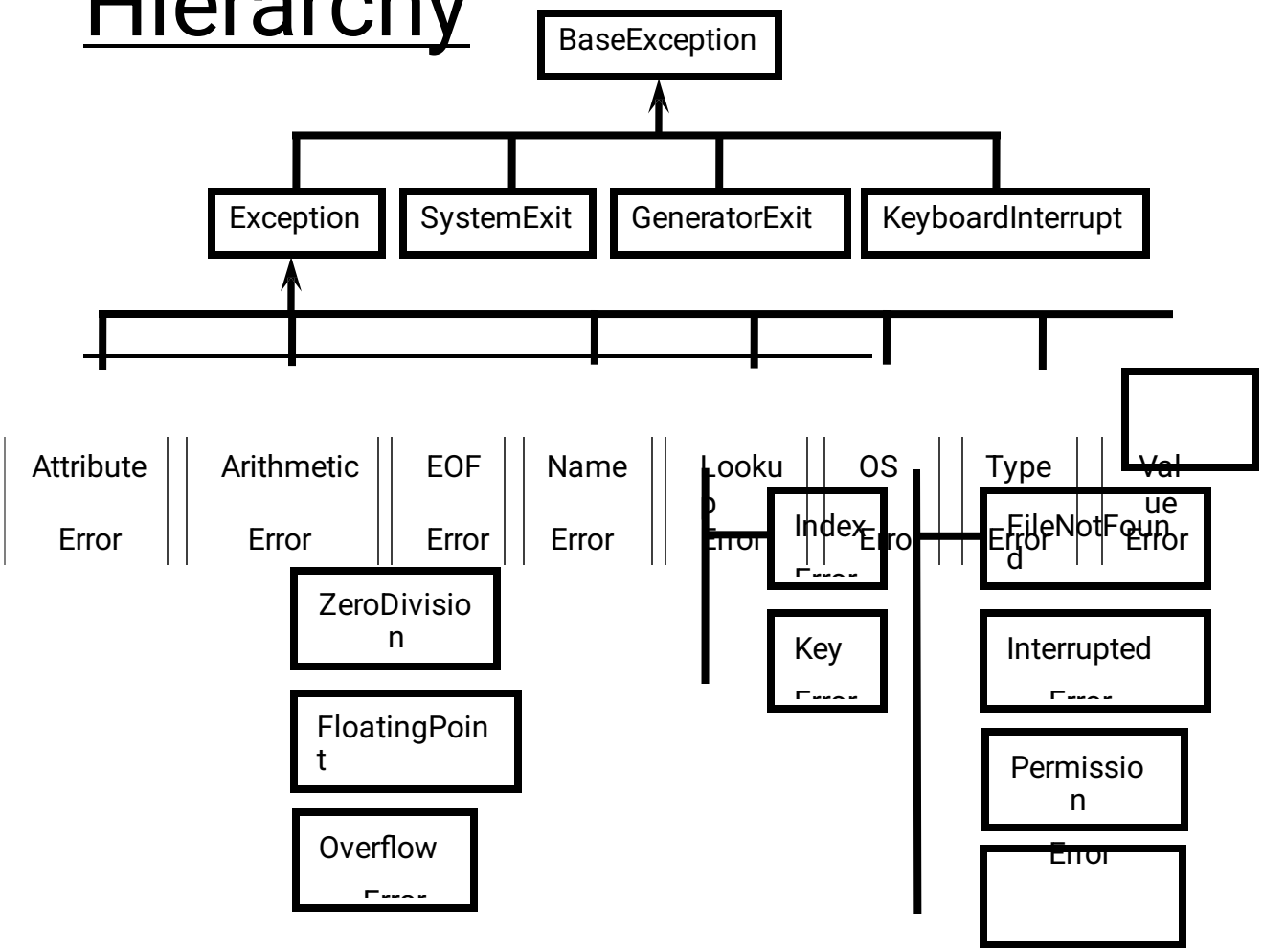
Whevever an exception occurs PVM will create the corresponding exception object and will check for handling code. If handling code is not available then Python interpreter terminates the program abnormally and prints corresponding exception information to the console.

The rest of the program won't be

executed. Eg:

```
1)  print("Hello")
2)  print(10/0)
3)  print("Hi")
4)
5)  D:\Python_classes>py test.py
6)  Hello
7)  Traceback (most recent call last):
8)     File "test.py", line 2, in <module>
9)       print(10/0)
10) ZeroDivisionError: division by zero
```

Python's Exception Hierarchy



Every Exception in Python is a class.

All exception classes are child classes of BaseException.i.e every exception class extends BaseException either directly or indirectly. Hence BaseException acts as root for Python Exception Hierarchy.

Most of the times being a programmer we have to concentrate Exception and its child classes.

Customized Exception Handling by using try-except:

It is highly recommended to handle exceptions.

The code which may raise exception is called risky code and we have to take risky

code inside try block. The corresponding handling code we have to take inside except block.

try:

Risky
Code
except
XXX:

Handling code/Alternative Code

without try-except:

```
print("stmt-1")
print(10/0)

print("stmt-3")
```

```
stmt-1
Output
```

Abnormal termination/Non-Graceful Termination

with try-except:

```
print("stmt-1")
try:
        print(10/0)
except ZeroDivisionError:
        print(10/2)
print("stmt-3")

Output
stmt-1
5.0
stmt-3
```

Normal termination/Graceful Termination

Control Flow in try-except:

try:

stmt-1

stmt-2

stmt-3

except XXX:

stmt-4
stmt-5

case-1: If there is no exception
1,2,3,5 and Normal
Termination

case-2: If an exception raised at stmt-2 and corresponding except block matched 1,4,5 Normal Termination

case-3: If an exception raised at stmt-2 and corresponding except block not matched 1, Abnormal Termination

case-4: If an exception raised at stmt-4 or at stmt-5 then it is always abnormal termination.

Conclusions:

1. within the try block if anywhere exception raised then rest of the try block wont be executed eventhough we handled that exception. Hence we have to take only risky code inside try block and length of the try block should be as less as possible.

2. In addition to try block,there may be a chance of raising exceptions inside except and finally blocks also.

3. If any statement which is not part of try block raises an exception then it is always abnormal termination.

How to print exception information:

try:

```
1.  print(10/0)
2. except ZeroDivisionError as msg:

4.      print("exception raised and its description is:",msg)
```

try with multiple except blocks:

The way of handling exception is varied from exception to exception.Hence for every exception type a seperate except block we have to provide. i.e try with multiple except blocks is possible and recommended to use.

Eg:

try:

```
except
    ZeroDivisionError:
    perform alternative
    arithmetic operations
```

except FileNotFoundError:

use local file instead of remote file

If try with multiple except blocks available then based on raised exception the corresponding except block will be executed.

Eg:

```
1)  try:
2)      x=int(input("Enter First Number: "))
3)                  y=int(input("Enter Second Number: "))
4)      print(x/y)
5)  except ZeroDivisionError :
6)      print("Can't Divide with Zero")
7)  except ValueError:
8)      print("please provide int value only")
9)
10) D:\Python_classes>py test.py
11) Enter First Number: 10
12) Enter Second Number: 2
13) 5.0
14)
15) D:\Python_classes>py test.py
16) Enter First Number: 10
17) Enter Second Number: 0
18) Can't Divide with Zero
19)
20) D:\Python_classes>py test.py
21) Enter First Number: 10
22) Enter Second Number: ten
23) please provide int value only
```

If try with multiple except blocks available then the order of these except blocks is important .Python interpreter will always consider from top to bottom until matched except block identified.

Eg:

```
1)  try:
2)      x=int(input("Enter First Number: "))
3)      y=int(input("Enter Second Number: "))
4)      print(x/y)
5)  except ArithmeticError :
6)      print("ArithmeticError")
7)  except ZeroDivisionError:
8)      print("ZeroDivisionError")
9)
10) D:\Python_classes>py test.py
```

```
11) Enter First Number: 10
12) Enter Second Number: 0
13) ArithmeticError
```

Single except block that can handle multiple exceptions:

We can write a single except block that can handle multiple different types of exceptions. except (Exception1,Exception2,exception3,..): or except (Exception1,Exception2,exception3,..) as msg :

Parenthesis are mandatory and this group of exceptions internally considered as tuple. Eg:

```
1)  try:
2)      x=int(input("Enter First Number: "))
3)                  y=int(input("Enter Second Number: "))
4)      print(x/y)
5)  except (ZeroDivisionError,ValueError) as msg:
6)      print("Plz Provide valid numbers only and problem is: ",msg)
7)
8)  D:\Python_classes>py test.py
9)  Enter First Number: 10
10) Enter Second Number: 0
11) Plz Provide valid numbers only and problem is: division by zero
12)
13) D:\Python_classes>py test.py
14) Enter First Number: 10
15) Enter Second Number: ten
16) Plz Provide valid numbers only and problem is: invalid literal for int() with b
17) ase 10: 'ten'
```

Default except block:

We can use default except block to handle any type of exceptions.

In default except block generally we can print normal error messages.

Syntax:

```
except:
    statements
```

Eg:

```
1)  try:
2)      x=int(input("Enter First Number: "))

4)      print(x/y)
```

```
3)              y=int(input("Enter Second Number: "))
```

```
except ZeroDivisionError:
    print("ZeroDivisionError:Can't divide with zero")
except:
    print("Default Except:Plz provide valid input only")

D:\Python_classes>py test.py
Enter First Number: 10
Enter Second Number: 0
ZeroDivisionError:Can't divide with zero

D:\Python_classes>py test.py
Enter First Number: 10
Enter Second Number: ten
Default Except:Plz provide valid input only
```

***Note: If try with multiple except blocks available then default except block should be last,otherwise we will get SyntaxError.

Eg:

```
try:
    print(10/0)
except:
    print("Default Except")
except ZeroDivisionError:
    print("ZeroDivisionError")

SyntaxError: default 'except:' must be last
```

Note:

The following are various possible combinations of except blocks

1. except ZeroDivisionError:
1. except ZeroDivisionError as msg:
3. except (ZeroDivisionError,ValueError) :
4. except (ZeroDivisionError,ValueError) as msg:
5. except :

finally block:

1. It is not recommended to maintain clean up code(Resource Deallocating Code or Resource Releasing code) inside try block because there is no guarentee for the execution of every statement inside try block always.

2. It is not recommended to maintain clean up code inside except block, because if there is no exception then except block won't be executed.

Hence we required some place to maintain clean up code which should be executed always irrespective of whether exception raised or not raised and whether exception handled or not handled. Such type of best place is nothing but finally block.

Hence the main purpose of finally block is to maintain clean up code.

try:

    Risky Code

except:

    Handling
Code finally:

    Cleanup code

The speciality of finally block is it will be executed always whether exception raised or not raised and whether exception handled or not handled.

Case-1: If there is no exception

```
1)  try:
2)      print("try")
3)  except:
4)      print("except")
5)  finally:
6)      print("finally")
7)
8)  Output
9)  try
10) finally
```

Case-2: If there is an exception raised but handled:

```
1)  try:
2)      print("try")
3)      print(10/0)
4)  except ZeroDivisionError:
5)      print("except")
6)  finally:
7)      print("finally")
8)
9)  Output
10) try
11) except
12) finally
```

Case-3: If there is an exception raised but not handled:

```
1) try:
2)     print("try")
3)     print(10/0)
4) except NameError:
5)     print("except")
6) finally:
7)     print("finally")
8)
9) Output
10) try
11) finally
12) ZeroDivisionError: division by zero(Abnormal Termination)
```

*** Note: There is only one situation where finally block won't be executed ie whenever we are using os._exit(0) function.

Whenever we are using os._exit(0) function then Python Virtual Machine itself will be shutdown.In this particular case finally won't be executed.

```
1) imports
2) try:
3)     print("try")
4)     os._exit(0)
5) except NameError:
6)     print("except")
7) finally:
8)     print("finally")
9)
10) Output
```

Note:

os._exit(0)

where 0 represents status code and it indicates normal termination There are multiple status codes are possible.

Control flow in try-except-finally:

try:

stmt-1

stmt-2

stmt-3

except:

finally:

stmt-5
stmt6

Case-1: If there is no exception
1,2,3,5,6 Normal Termination

Case-2: If an exception raised at stmt2 and the corresponding except block matched
1,4,5,6 Normal Termination

Case-3: If an exception raised at stmt2 but the corresponding except block not matched 1,5 Abnormal Termination

Case-4:If an exception raised at stmt4 then it is always abnormal termination but before that finally block will be executed.

Case-5: If an exception raised at stmt-5 or at stmt-6 then it is always abnormal termination

Nested try-except-finally blocks:

We can take try-except-finally blocks inside try or except or finally blocks.i.e nesting of try- except-finally is possible.

try:

_____

_____

_____

except:

_____

_____

_____

_____

except:

_____

_____

_____

General Risky code we have to take inside outer try block and too much risky code we have to take inside inner try block. Inside Inner try block if an exception raised then inner

except block is responsible to handle. If it is unable to handle then outer except block is responsible to handle.

Eg:

```
1)  try:
2)      print("outer try block")
3)      try:
4)          print("Inner try block")
5)          print(10/0)
6)      except ZeroDivisionError:
7)          print("Inner except block")
8)      finally:
9)          print("Inner finally block")
10) except:
11)     print("outer except block")
12) finally:
13)     print("outer finally block")
14)
15) Output
16) outer try block
17) Inner try block
18) Inner except block
19) Inner finally block
20) outer finally block
```

Control flow in nested try-except-finally:

finally:

stmt-9 except Y:

stmt-8

finally:

stmt-10

stmt-11

case-1: If there is no exception
1,2,3,4,5,6,8,9,11,12 Normal Termination

case-2: If an exception raised at stmt-2 and the corresponding except block matched
1,10,11,12 Normal Termination

case-3: If an exceptiion raised at stmt-2 and the corresponding except block not matched 1,11,Abnormal Termination

case-4: If an exception raised at stmt-5 and inner except block matched
1,2,3,4,7,8,9,11,12 Normal Termination

case-5: If an exception raised at stmt-5 and inner except block not matched but outer except block matched

1,2,3,4,8,10,11,12,Normal Termination

case-6:If an exception raised at stmt-5 and both inner and outer except blocks are not matched

1,2,3,4,8,11,Abnormal Termination

case-7: If an exception raised at stmt-7 and corresponding except block matched
1,2,3,.,.,.,8,10,11,12,Normal Termination

case-8: If an exception raised at stmt-7 and corresponding except block not matched
1,2,3,.,.,.,8,11,Abnormal Termination

case-9: If an exception raised at stmt-8 and corresponding except block matched
1,2,3,.,.,.,.,10,11,12 Normal Termination

case-10: If an exception raised at stmt-8 and corresponding except block not matched
1,2,3,.,.,.,.,11,Abnormal Termination

case-11: If an exception raised at stmt-9 and corresponding except block matched
1,2,3,.,.,.,.,8,10,11,12,Normal Termination

case-12: If an exception raised at stmt-9 and corresponding except block not matched
1,2,3,.,.,.,.,8,11,Abnormal Termination

case-13: If an exception raised at stmt-10 then it is always abonormal termination but before abnormal termination finally block(stmt-11) will be executed.

case-14: If an exception raised at stmt-11 or stmt-12 then it is always abnormal termination.

Note: If the control entered into try block then compulsary finally block will be executed. If the control not entered into try block then finally block won't be executed.

else block with try-except-finally:

We can use else block with try-except-finally blocks.

else block will be executed if and only if there are no exceptions inside try block.

```
try:
        Risky Code
except:
        will be executed if exception inside try
```

else: finally:

print("else") print("finally")

If we comment line-1 then else block will be executed b'z there is no exception inside try. In this case the output is:

try
else
finally

If we are not commenting line-1 then else block won't be executed b'z there is exception inside try block. In this case output is:

try
except

finally

Various possible combinations of try-except-else-finally:

1. Whenever we are writing try block, compulsory we should write except or finally block.i.e without except or finally block we cannot write try block.

2. Wheneever we are writing except block, compulsory we should write try block. i.e except without try is always invalid.

3. Whenever we are writing finally block, compulsory we should write try block. i.e finally without try is always invalid.

4. We can write multiple except blocks for the same try,but we cannot write multiple finally blocks for the same try

5. Whenever we are writing else block compulsory except block should be there. i.e without except we cannot write else block.

6. In try-except-else-finally order is important.

7. We can define try-except-else-finally inside try,except,else and finally blocks. i.e nesting of try-except-else-finally is always possible.

1
```
try:
    print("try")
```

2
```
except:
    print("Hello")
```

3
```
else:
    print("Hello")
```

4
```
finally:
    print("Hello")
```

5
```
try:
    print("try")
except:
    print("except")
```
√

6
```
try:
    print("try")
finally:
    print("finally")
```
√

7
```
try:
    print("try")
```
√

```
except:
        print("except")
else:
        print("else")
```

```
try:
        print("try")
else:
        print("else")
```

8

```
try:
        print("try")
else:
        print("else")
finally:
        print("finally")
```

9

```
try:
      print("try")
except XXX:
      print("except-1
") except YYY:
      print("except-2")
```

10 √

```
try:
        print("try")
except :
     print("except-1")
 else:
        print("else")
else:
        print("else")
```

11

```
try:
        print("try")
except :
        print("except-1")
finally:
        print("finally")
finally:
        print("finally")
```

12

```
try:
        print("try")
        print("Hello")
except:
        print("except")
```

13

```
try:
        print("try")
except:
```

14

```
        print("except"
) print("Hello")
except:
        print("except")
try:
        print("try")
except:
15      print("except"
) print("Hello")
finally:
        print("finally")
try:
        print("try")
except:
16      print("except"
) print("Hello")
else:
        print("else")
try:
        print("try")
except:
17      print("except")                                    √
try:
        print("try")
except:
        print("except")
try:
        print("try")
except:
18      print("except")                                    √
try:
        print("try")
finally:
        print("finally")
try:
        print("try")
except:
19      print("except"
) if 10>20:
        print("if")
else:
        print("else")
try:
20                                                         √
        print("try")
```

```
            try:
                print("inner
          try") except:
                print("inner except
            block") finally:
        print("inner finally block")
    except:
            print("except")
    try:
            print("try")

    except:
            print("except"
21          ) try:                                  √
                print("inner
            try") except:
                print("inner except
            block") finally:
        print("inner finally block")
    try:
            print("try")
    except:
            print("except")
    finally:
22          try:                                    √
            print("inner
            try") except:
                print("inner except
            block") finally:
                print("inner finally
                block")
    try:
            print("try")
    except:
            print("except")
23  try:
            print("try")
    else:
            print("else")
    try:
            print("try")
            try:
24              print("inner try")
    except:
            print("except")
```

```
try:
        print("try")
else:
        print("else")
except:
        print("except")
finally:
        print("finally")
```

25

Types of Exceptions:

In Python there are 2 types of exceptions are possible.

1. Predefined Exceptions
2. User Definded Exceptions

1. Predefined Exceptions:

Also known as in-built exceptions

The exceptions which are raised automatically by Python virtual machine whenver a particular event occurs, are called pre defined exceptions.

Eg 1: Whenever we are trying to perform Division by zero, automatically Python will raise ZeroDivisionError.

```
print(10/0)
```

Eg 2: Whenever we are trying to convert input value to int type and if input value is not int value then Python will raise ValueError automatically

```
x=int("ten")===>ValueError
```

2. User Defined Exceptions:

Also known as Customized Exceptions or Programatic Exceptions

Some time we have to define and raise exceptions explicitly to indicate that something goes wrong ,such type of exceptions are called User Defined Exceptions or Customized Exceptions

Programmer is responsible to define these exceptions and Python not having any idea about these. Hence we have to raise explicitly based on our requirement by using "raise" keyword.

Eg:
InSufficientFundsExcepti
on InvalidInputException
TooYoungException
TooOldException

How to Define and Raise Customized Exceptions:

Every exception in Python is a class that extends Exception class either directly or indirectly.

Syntax:

```
class classname(predefined exception class
    name): def _init_(self,arg):
            self.msg=arg
```

Eg:

```
1) class TooYoungException(Exception):
2)     def _init_(self,arg):
3)                 self.msg=arg
```

TooYoungException is our class name which is the child class of

Exception We can raise exception by using raise keyword as follows

raise TooYoungException("message")

Eg:

```
1)  class
2)     def _init_(self,arg):
3)         self.msg=ar
4)
5)  class TooOldException(Exception):
6)     def _init_(self,arg):
7)         self.msg=ar
8)
9)  age=int(input("Enter Age:"))
10) if age>60:

12) elif age<18:



14) else:
    getting ma rriage")
16)
15)  print("You will get match details soon by email!!!")
```

```
Enter Age:90
__main__.TooYoungException: Plz wait some more time you will get best match soon!!!

D:\Python_classes>py test.py
Enter Age:12
__main__.TooOldException: Your age already crossed marriage age...no chance of g
etting marriage

D:\Python_classes>py test.py
Enter Age:27
You will get match details soon by email!!!
```

Note:

raise keyword is best suitable for customized exceptions but not for pre defined exceptions